

Eliminarea partilor nevizibile ale scenelor 3D din imagini - 1

Prof. univ. dr. ing. Florica Moldoveanu

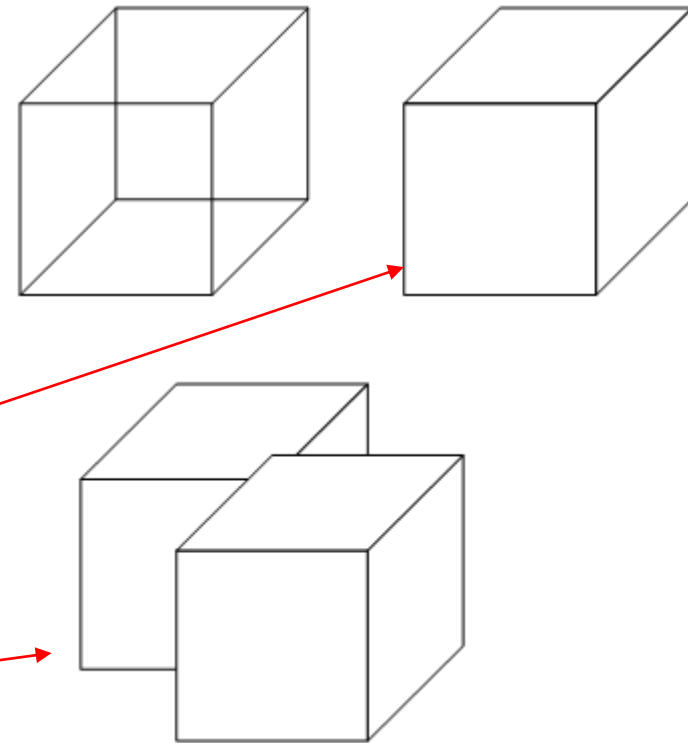
Eliminarea partilor nevizibile ale scenelor 3D, din imagini (1)

Vizibilitatea obiectelor dintr-o scena 3D, într-o imagine, depinde de:

- Pozitia observatorului si directia in care priveste
- Pozitionarea obiectelor din scena unul față de celalalt
- Volumul vizual

Partile unei scene 3D nevizibile într-o imagine sunt:

- Părți aflate total sau parțial în afara volumului vizual
- Fețe auto-obturate: fețele unui obiect obturate de celelalte parti ale aceluiasi obiect
- Părți obturate (ascunse) de obiecte ale scenei, aflate în fața lor în raport cu observatorul



Eliminarea părților nevizibile ale scenelor 3D, din imagini (2)

- ❖ Eliminarea părților dintr-o scenă 3D care nu sunt nevizibile într-o imagine este efectuată la mai multe niveluri:

1. Eliminarea obiecte sau grupuri de obiecte: grupuri de primitive - Object culling:

- **Frustum Culling:** eliminarea din banda grafică a grupurilor de primitive care sunt complet în afara volumului vizual.
- **Occlusion Culling:** – eliminarea din banda grafică a grupurilor de primitive care sunt complet obturate de alte grupuri de primitive (obiecte).

- nu este efectuată de OpenGL, nici de GPU
- poate fi efectuată prin algoritmi implementați într-o bibliotecă folosită de aplicația grafică 3D (motorul grafic).

2. Eliminarea fețelor auto-obturate ale obiectelor - *Back face culling*:

- este efectuată de GPU.

Eliminarea părților nevizibile ale scenelor 3D, din imagini (3)

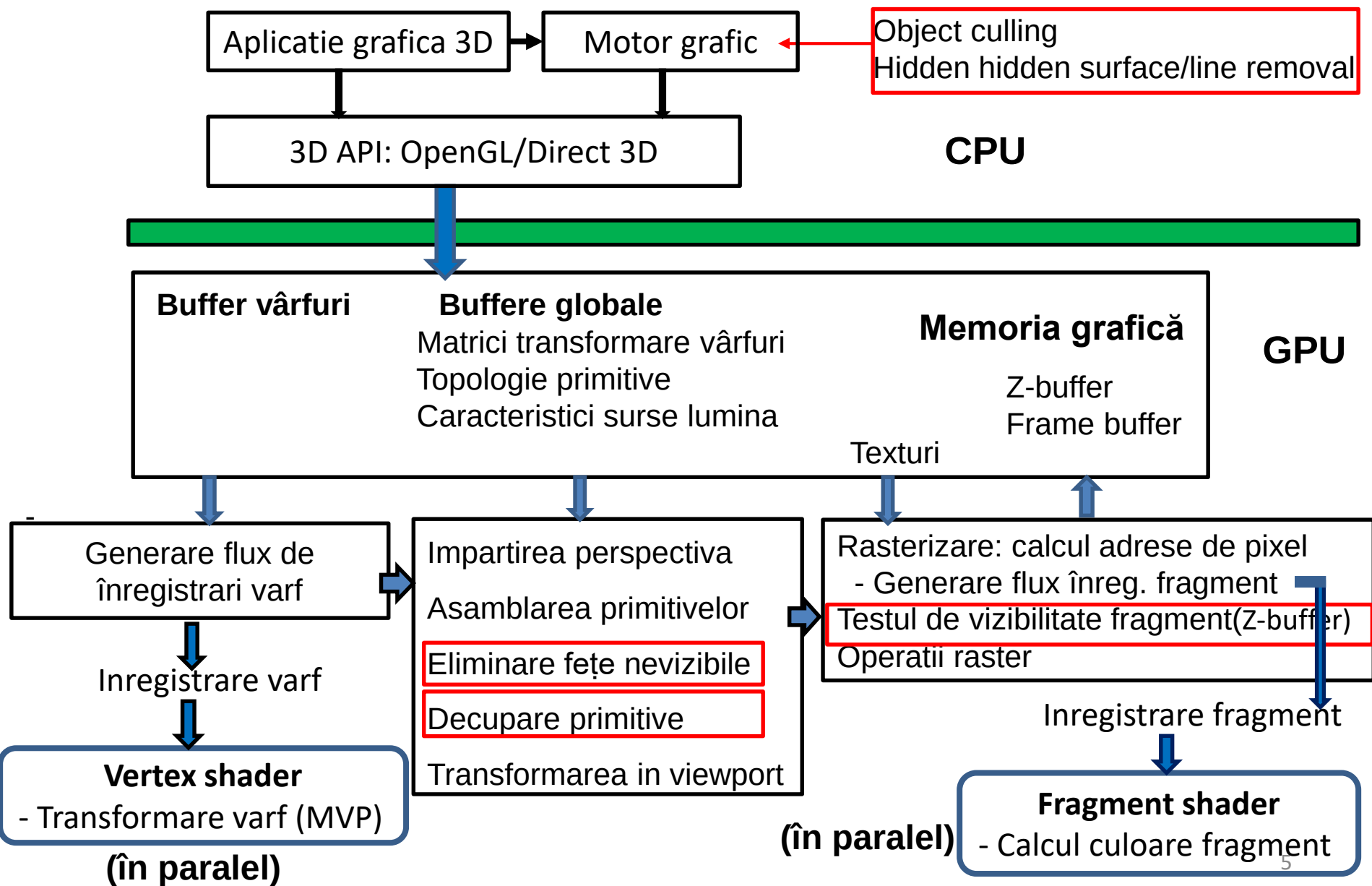
3. Eliminarea părților nevizibile ale primitivelor grafice (poligoane, linii):

- Eliminarea primitivelor/partilor de primitive aflate în afara volumului vizual, prin operatia de **decupare (clipping)** efectuada de GPU
- Eliminarea părților obturate (ascunse) de alte primitive grafice: ***hidden surface/hidden line removal***
 - poate fi efectuada prin algoritmi implementati în motorul grafic folosit de aplicație: algoritmul BSP, algoritmul Pictorului, ș.a.

4. Eliminarea fragmentelor nevizibile

- este efectuada de GPU: fragmentele de primitive obturate de alte fragmente (aflate mai aproape de observator) sunt suprascrise la momentul rasterizarii (algoritmul Z-buffer)

Eliminarea partilor nevizibile in banda grafica

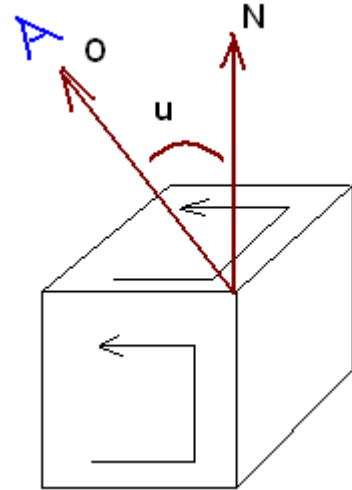


Eliminarea fețelor auto-obturate pentru poliedre convexe (Back face culling)(1)

- ❖ Poate reduce substantial numărul de poligoane rasterizate.
- ❖ Clasificarea fețelor unui obiect în “fețe din față (vizibile)” și “fețe din spate” se face pe baza poziției obiectului față de observator

Algoritmul se bazează pe următoarele presupuneri:

1. Observatorul și obiectul sunt raportați la același sistem de coordonate.
2. Conturul fiecărei fețe a obiectului este orientat în sens trigonometric atunci când poliedrul este văzut din exterior; condiția asigură ca normala fiecărei fețe văzută din exterior este orientată către spațiul exterior obiectului.
3. Observatorul este situat în afara obiectului.



O față este vizibilă dacă : $0 \leq u < 90^\circ$ sau $0 < \cos(u) \leq 1$

u: unghiul dintre normala la față și vectorul orientat către observator

Produsul scalar: $N \cdot O = |N| * |O| * \cos(u) \rightarrow \cos(u) = N \cdot O / (|N| * |O|) = N_u \cdot O_u$

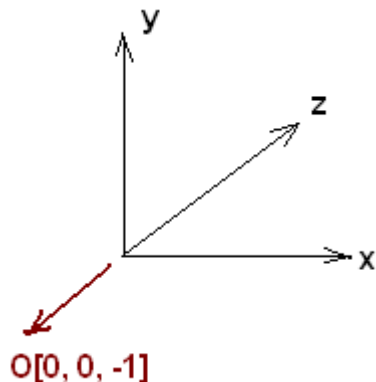
Condiția de vizibilitate: $N_u \cdot O_u > 0$ (produsul scalar al versorilor)

Determinarea fețelor auto-obturate pentru poliedre convexe(2)

Determinarea fețelor auto-obturate poate fi efectuată:

1. In sistemul coordonatelor globale (in care este definit si observatorul – camera virtuala)
2. In sistemul de coordonate observator
3. In sistemul coordonatelor de decupare, transformand pozitia observatorului cu matricea

V•P. In acest caz:



Sistemul coordonatelor de decupare

$$N_u \bullet O_u = [n_x, n_y, n_z] \bullet [0, 0, -1] = -n_z$$

Conditia de vizibilitate devine: $n_z < 0$
(N si O au aceeasi orientare)

Dupa transformarea de proiectie pozitia observatorului este la infinit, pe axa z negativa.

Vectorul orientat catre observator este acelasi pentru orice punct al unui obiect.

Determinarea fetelor auto-obturate pentru poliedre convexe(3)

Calculul normalei la o față a poliedrului

Fie $V_1(x_1, y_1, z_1)$, $V_2(x_2, y_2, z_2)$, $V_3(x_3, y_3, z_3)$, 3 varfuri succesive ale conturului feței. Normala la față se poate obtine calculand produsul vectorial: $(V_1-V_2) \times (V_2-V_3)$

$$\mathbf{N} = (V_1-V_2) \times (V_2-V_3) = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ x_2-x_1 & y_2-y_1 & z_2-z_1 \\ x_3-x_2 & y_3-y_2 & z_3-z_2 \end{vmatrix} =$$

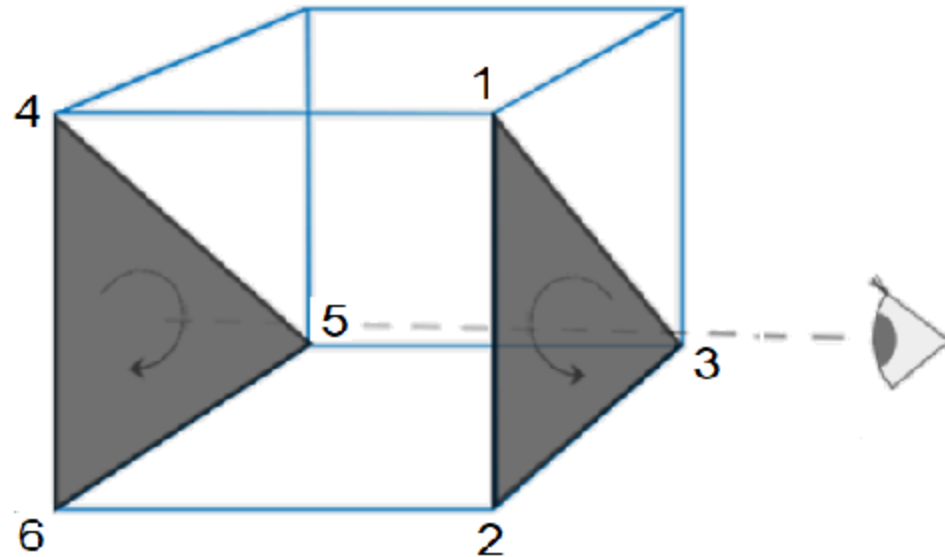
$$[(z_3-z_2)*(y_2-y_1) - (z_2-z_1)*(y_3-y_2)]*\mathbf{i} - [(x_2-x_1)*(z_3-z_2) - (x_3-x_2)*(z_2-z_1)]*\mathbf{j} \\ + [(x_2-x_1)*(y_3-y_2) - (x_3-x_2)*(y_2-y_1)]*\mathbf{k}$$

unde $\mathbf{i}$, $\mathbf{j}$, $\mathbf{k}$ sunt versorii directiilor axelor sistemului de coordonate cartezienne 3D

Determinarea fetelor nevizibile pe GPU (1)

- Presupunem ca pentru toate fețele unui cub s-a definit in program orientarea trigonometrica a conturilor lor, atunci and sunt vazute din exteriorul cubului: 1,2,3; 4,5,6, etc.
- Dupa transformarea de proiectie, in raport cu pozitia observatorului, ordinea varfurilor fețelor proiectate este:

- trigonometrica pentru fețele “din fata” (vizibile); ex. 1,2,3;
- anti-trigonometrica pentru fețele “din spate”; ex. ordinea 4,5,6 din pozitia observatorului este anti-trigonometrica – corespunde feței vazute din interior



Determinarea fetelor nevizibile pe GPU (2)

OpenGL da posibilitatea programatorului sa specifice care este orientarea fetelor “din față” și care fațe trebuie sa fie eliminate (sunt nevizibile).

1. Pentru activarea operatiei de eliminare a fetelor nevizibile (“face culling”) se apeleaza:

`glEnable(GL_CULL_FACE);` //implicit, operatia este dezactivata

2. Pentru a specifica orientarea fetelor “din față”: trigonometrica (GL_CCW)/sensul acelor de ceas (GL_CW):

`glFrontFace(GL_CCW/GL_CW);`

3. Pentru a specifica fețele care se dorește a fi eliminate: din față / din spate/toate

`glCullFace(GL_FRONT/ GL_BACK/GL_FRONT_AND_BACK);`

În cazul **GL_FRONT_AND_BACK** nu vor fi afisate fețe, ci numai alte primitive: puncte, linii.

Exemplu: dacă toate fețele unui obiect sunt orientate trigonometric, apelând

`glCullFace (GL_BACK)` și `glFrontFace(GL_CCW)` sau `glCullFace (GL_FRONT)` și `glFrontFace(GL_CW)`

se elimina fețele ale caror proiectii nu sunt orientate trigonometric.

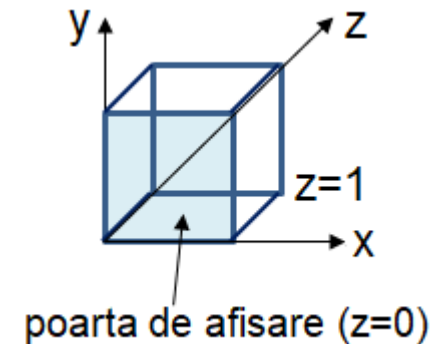
Algoritmul z-buffer(1)

- Algoritm de eliminare a fragmentelor nevizibile ale primitivelor grafice.
- Executat de GPU, fiind integrat in procesul de rasterizare a primitivelor

❖ Primitivele rasterizate au coordonatele raportate la spatiul ecran

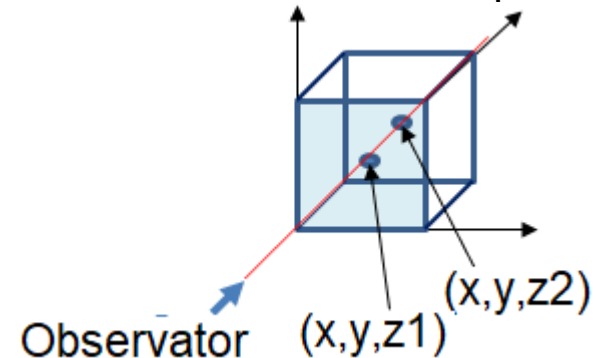
❖ Observatorul este la infinit, pe axa z negativa

❖ Asupra primitivelor se efectueaza proiectie ortografica
in planul XOY (poarta de afisare)



Daca doua fragmente aflate pe acelasi proiector, rezultate din rasterizarea a doua primitive diferite, au coordonata z diferita:

➤ Fragmentul avand coordonata z mai mica va fi afisat în pixelul (x,y).



Algorítmul z-bufer(2)

- Primitivele grafice (triunghiuri, linii) în care a fost descompusa scena 3D sunt rasterizate în ordinea în care au fost transmise din programul de aplicație (nu sunt sortate).

- Buffer-ul imagine (frame buffer) este actualizat pe măsura rasterizării primitivelor, astfel:

dacă (fragmentul curent se proiectează în pixelul (x,y) și

$z_{\text{fragment}} < z_{\text{fragment afisat în pixelul } (x,y)}$)

atunci

actualizează culoarea pixelului (x,y) în bufer-ul imagine, la culoarea fragmentului curent

Rezultă: este necesar să se memoreze coordonatele z ale fragmentelor afisate în pixelii imaginii



Z-buffer: - tablou bidimensional cu număr de elemente egal cu numărul de pixeli ai suprafeței de afisare

- în memoria grafică

Algorítmul z-buffer(3)

Algorítmul z-buffer (integrat in procesul de rasterizare):

- *Initializeaza buffer-ul imagine (frame buffer) la culoarea de fond
- *Initializeaza Z-buffer la coordonata z maxima ($z=1$)

Pentru fiecare fragment $f(x,y,z)$ rezultat din rasterizarea unei primitive

- * calculeaza culoarea fragmentului (in fragment shader)

//testul de vizibilitate (adancime)

daca $z < Z\text{-buffer}[y][x]$ atunci

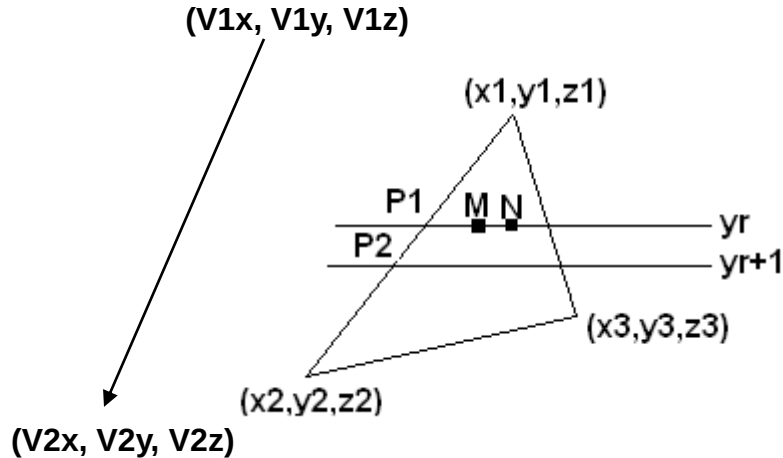
{ $Z\text{-buffer}[y][x] = z$

*actualizeaza culoarea pixelului (x,y) in buffer-ul imagine la culoarea fragmentului f

}

Algoritmul z-bufer(4)

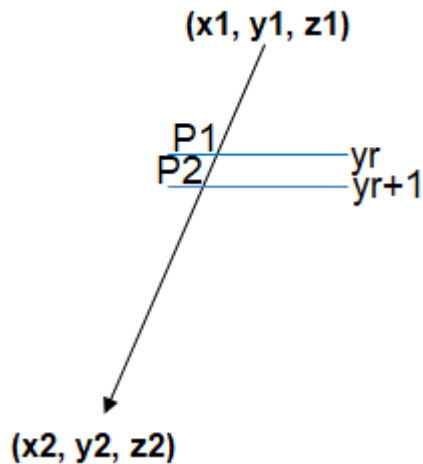
Calculul coordonatei z a unui fragment



Coordonatele z ale fragmentelor rezultate la rasterizarea vectorilor si a suprafetelor triunghiulare se obtin prin calcul incremental, in algoritmul de rasterizare.

Algoritmul z-bufer(5)

Calculul coordonatei z la rasterizarea vectorilor



$$x = x1 + t(x2-x1)$$

$$y = y1 + t(y2-y1) \quad \text{Ecuatiile parametrice ale vectorului}$$

$$z = z1 + t(z2-z1)$$

y_r, y_{r+1} : ordonatele a 2 linii raster (ecran) care intersecteaza vectorul

$$y(P1) = y_r = y1 + t'(y2-y1) \rightarrow t' = (y_r - y1)/(y2-y1)$$

$$z(P1) = z1 + (y_r - y1)(z2 - z1)/(y2 - y1) = z1 + (y_r - y1) * K,$$

K este o constanta la rasterizarea vectorului

$$z(P2) = z1 + (y_{r+1} - y1) * K = z1 + (y_r - y1) * K + K$$

$z(P2) = z(P1) + K \rightarrow$ coordonatele z ale fragmentelor de pe vector se obtin prin calcul incremental.

Algoritmul z-bufer(6)

Calculul coordonatei z la rasterizarea poligoanelor

$P1(xP1, yr, zP1), P2(xP2, yr+1, zP2)$

m- panta laturii $(x1,y1,z1) - (x2,y2,z2)$

$xP2 = xP1 + 1/m$ (vezi curs - rasterizare)

$A*x + B*y + C*z + D = 0$ ec planului poligonului

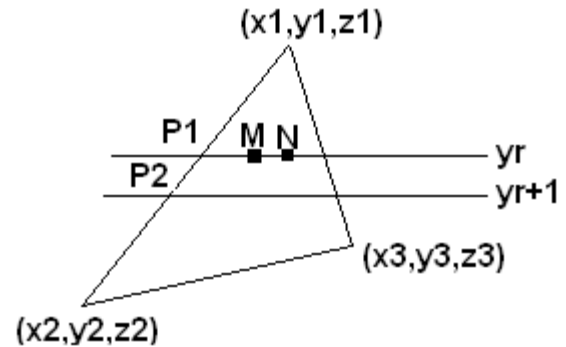
$zP1 = (-A*xP1 - B*yr - D)/C$

$zP2 = (-A*xP2 - B*(yr+1) - D)/C = (-A(xP1 + 1/m) - B*yr - B - D)/C = zP1 + (-A/m - B) \rightarrow zP2 = zP1 + K1$

$M(xM, yr, zM), N(xM + 1, yr, zN)$

$zN = (-A*(xM + 1) - B*yr - D)/C = zM + (-A/C) \rightarrow zN = zM + k2$

Observatie: atunci cand un vector se suprapune pe suprafata unui poligon pot aparea defecte datorita modului diferit in care se calculeaza coordonatele z ale fragmentelor care se afiseaza in aceiasi pixeli!



Algoritmul z-bufer(7)

Aprecieri asupra algoritmului z-Buffer

- 1) Numarul de comparatii de valori z pentru un pixel = numarul de fragmente care se proiecteaza in acel pixel.

Timpul de calcul tinde sa devina independent de numarul de poligoane: in medie, numarul de pixeli acoperiti de un poligon este invers proportional cu numarul de poligoane.

- 2) Bufferul Z este un buffer de numere intregi: pentru o precizie buna, este necesara o reprezentare pe 32 biți a valorilor din z-buffer.
- 3) “Depth fighting”: doua fragmente care se afiseaza in acelasi pixel, cu z diferit in spatiul observator, au acelasi z in Z-buffer → culoarea pixelului depinde de ordinea rasterizarii primitivelor din care fac parte cele 2 fragmente
- 4) “Depth complexity”: numarul de suprascrieri ale unui pixel in buffer-ul imagine, la generarea unui cadru imagine (presupunand calcul culoare fragment, utilizare texturi)
 - poate fi redusa prin “deffered rendering”: se calculeaza culoarea numai pentru fragmentele vizibile in pixelii imaginii.

Eliminarea partilor nevizibile ale scenelor 3D din imagini -2

Prof. univ. dr. ing. Florica Moldoveanu

Algoritmul BSP pentru eliminarea părților nevizibile ale primitivelor

- ❖ Scena 3D este reprezentată printr-un arbore binar, denumit sugestiv arbore BSP (**Binary Space Partitioning**)
- ❖ Inițial, arborele BSP a fost folosit pentru operația de *object culling*.
- ❖ Ulterior a fost folosit și ca algoritm pentru eliminarea părților nevizibile ale primitivelor grafice (hidden surface removal), fiind folosit în acest scop în jocuri ca *Doom* și *Quake*, pentru a crește viteza de redare a scenelor 3D.
- ❖ Algoritmul este eficient pentru eliminarea părților nevizibile ale poligoanelor. Eliminarea părților nevizibile ale liniilor se rezolvă eficient la nivel de fragment.
- **Arborele BSP al unei scene este independent de poziția observatorului - reprezintă scena în sistemul coordonatelor globale.**

Intrarea algoritmului de construire a arborelui: lista poligoanelor care compun scena 3D, în orice ordine; nu are importanță din care obiect face parte fiecare poligon.

Construirea arborelui BSP (1)

Construirea arborelui BSP al unei scene

- Fiecare nod al arborelui corespunde unui plan de partitionare a spatiului 3D (de regula, planul unui poligon al scenei)

- Fiecare plan de partitionare împarte spatiul în 2 semi-spacii:
 - cel din fața planului (de aceeași parte cu normala la plan)
 - cel din spatele planului

1. Se începe cu un poligon oarecare din lista (de regula primul),

pentru care se crează nodul rădăcină al arborelui

- Poligoanele aflate în semi-spaciul “față” formează

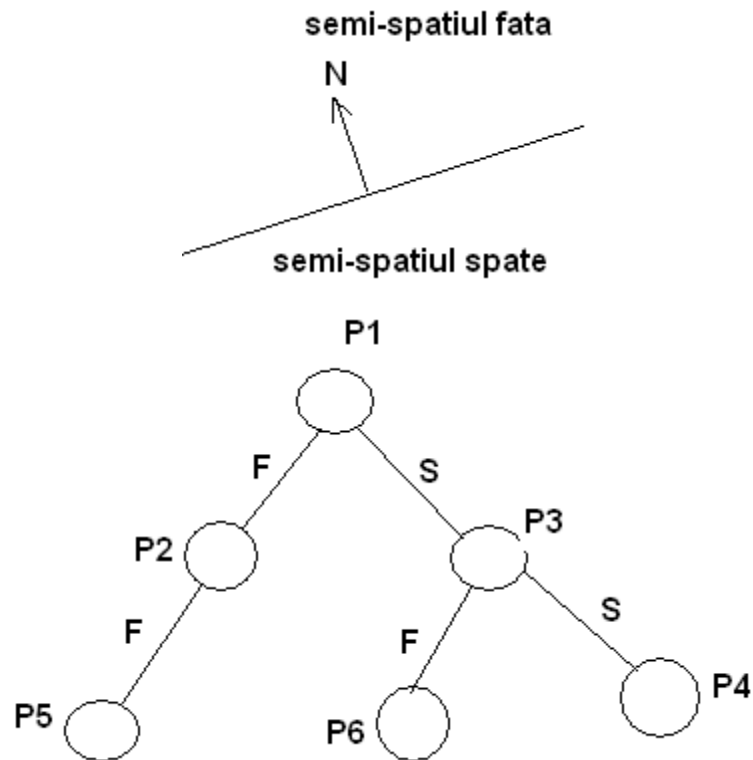
“lista-față”, care va genera subarborii “față” al nodului

- Primitivele din semi-spaciul “spate” formează “lista-spate”,

care va genera subarborii “spate” al nodului.

2. Se alege un poligon din lista-față și se crează nodul rădăcină al subarborii “față,

etc.



Construirea arborelui BSP (2)

Algoritmul de creare a arborelui BSP:

arbore * creareBSP (Poligon * LP)

{ Poligon P, * ListaFata, * ListaSpate, * ListaNod;

daca (LP este vida) return NULL;

ListaFata = ListaSpate = ListaNod = NULL;

* alege un poligon P din LP;

* elimina P din LP; adauga P la ListaNod;

pentru (fiecare poligon Q din LP) executa

{ **daca** (Q este in semispatiul față al planului lui P) **atunci**

* adauga Q in ListaFata

altfel

Construirea arborelui BSP (3)

daca (Q este in semispatiul spate al planului lui P) **atunci**

- * adauga Q in ListaSpate

altfel

daca Q este in acelasi plan cu P **atunci**

- * adauga Q in ListaNod

altfel // Q este intersectat de planul lui P

- * divizeaza Q cu planul lui P

- * adauga fiecare poligon rezultat din intersectie in ListaFata sau ListaSpate,
in functie de pozitia sa

} // pentru fiecare poligon

arbore * radacina = combina(creareBSP(ListaFata), creareBSP(ListaSpate));

*ataseaza ListaNod la radacina;

return radacina;

}

Afișarea scenei reprezentata prin arbore BSP

❖ Tine cont de pozitia observatorului

- Afișarea “**back-to-front**” - poligoanele sunt trimise in banda grafica in ordinea: de la cel mai indepartat de observator pana la cel mai apropiat de observator
- Se porneste din radacina arborelui si se avanseaza in arbore pana la frunze, in fiecare nod coborandu-se in subarborele nodului care se afla de partea opusa observatorului fata de planul nodului.

void afisareBSP_back_to_front(arbore * A, Pozitie Observator)

{ **daca** (! A) **return**;

daca (Observator este in semispatiul fata al planului radacinii arborelui A) **atunci**

 { afisareBSP(A->spate); *afisare poligoane din nodul radacina; afisareBSP(A->fata);}

altfel

 { afisareBSP(A->fata); *afisare poligoane din nodul radacina; afisareBSP(A->spate);}

}

Afișarea scenei reprezentata prin arbore BSP

- **Afișarea “front-to-back”** - poligoanele sunt trimise in banda grafica in ordinea: de la cel mai apropiat de observator pana la cel mai îndepărtat de observator.
- Se porneste din radacina arborelui si se avanseaza in arbore pana la frunze, in fiecare nod coborandu-se in subarborele nodului care se afla de aceeasi parte cu observatorul fata de planul nodului.

void afisareBSP_front_to_back(arbore * A, Pozitie Observator)

{ **daca** (! A) **return**;

daca (Observator este in semispatiul fata al planului radacinii arborelui A) **atunci**

 { afisareBSP(A->fata); *afisare poligoane din nodul radacina; afisareBSP(A->spate);}

altfel

 { afisareBSP(A->spate); *afisare poligoane din nodul radacina; afisareBSP(A->fata);}

}

Algoritmul BSP - aprecieri (1)

- Arborele nu trebuie sa fie reconstruit pentru fiecare cadru imagine, daca scena nu s-a modificat: avantaj pentru scenele statice
- Modificarea pozitiei observatorului → executia functiei de afisare (daca scena nu s-a modificat)
- Pentru obiectele dinamice se pot face diverse adaptari ale algoritmului; de ex:
 - pentru fiecare cadru imagine se insereaza in arborele BSP cu obiectele statice, noduri corespunzand obiectelor dinamice, in functie de pozitia fiecaruia;
 - se construiesc arborele BSP numai cu obiectele statice iar obiectele dinamice sunt trimise direct in banda grafica, eliminarea partilor obturate de acestea fiind realizata la rasterizare.
- **In cazul afisarii back-to front**, poligoanele mari (ex. pereti) din spate sunt afisate primele si apoi părți mari din ele sunt obturate de poligoanele mai apropiate de observator, prin suprascrierea pixelilor → timp consumat inutil cu calculul culorilor fragmentelor poligoanelor din spate.

Algoritmul BSP – aprecieri (2)

- **In cazul afisarii front-to back**, poligoanele apropiate de observator sunt afisate primele, apoi cele din spate sunt “decupate” de poligoanele deja afisate, prin testul de adancime (pixelii acoperiti de poligoanele din fata nu sunt suprascrisi). In plus prin “early depth test”, o optiune implementata de placile grafice recente, se evita calculul culorii fragmentelor care nu trec testul de adancime.
- Deoarece nu au loc suprascrieri de pixeli, majoritatea motoarelor grafice care folosesc BSP utilizeaza afisarea front-to-back.
- **In cazul afisarii back-to front** nu mai este necesar testul de adâncime la afisarea fragmentelor (fragmentele care se proiecteaza in acelasi pixel sunt suprascrise), de aceea se poate dezactiva testul de adâncime:

`glEnable(GL_DEPTH_TEST) / glDisable(GL_DEPTH_TEST)`
- **Arborele BSP este foarte eficient daca se folosește pentru operatia *object culling***
- Construirea si afisarea arborelui BSP sunt functii de management al scenei 3D, implementate de regula intr-un motor grafic 3D.

Modele de reflexie -1

Prof. univ. dr. ing. Florica Moldoveanu

Curs Elemente de Grafică pe Calculator – UPB, Automatică și Calculatoare
2019-2020

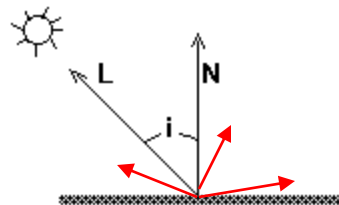
Modele empirice pentru calculul reflexiei luminii

- Caracteristicile luminii reflectate într-un punct al suprafeței unui obiect depind de :
 - lungimile de undă conținute în lumina incidentă,
 - direcția și geometria sursei luminii incidente,
 - orientarea suprafeței în punctul respectiv,
 - proprietățile materialului din care este construită suprafața.
- Expresia care modelează intensitatea luminii reflectate într-un punct al unei suprafețe este definită pentru o **lumină incidentă monocromatică**, deci caracterizată printr-o lungime de undă, λ .
- În cazul general, **lumina incidentă nu este monocromatică**, de aceea pentru calculul său expresia ar trebui să fie evaluată continuu pe întregul domeniu al spectrului luminii incidente.
- În practică, expresia se evaluează separat pentru cele trei culori primare, R, G, B.

Modelarea reflexiei difuze a luminii provenite de la o sursa

Aproximarea reflexiei difuze intr-un punct al unei suprafete 3D

- Lumina reflectată difuz de o suprafață este dispersată regulat în toate direcțiile.
- **Legea lui Lambert** definește reflexia luminii provenite de la o sursă punctiformă, de către un difuzor perfect:



Reflexia difuza

$$I_d = I_{\text{sursa}} * k_d * \cos(i) \quad 0 \leq i \leq \pi/2$$

N – normala la suprafata in punctul considerat

I_{sursa} – este intensitatea luminii incidente (provenita de la sursa de lumina)

k_d – este coeficientul de difuzie a luminii incidente, dependent de materialul suprafetei: suprafata reflecta/absoarbe/transmite lumina in anumite proportii: $0 \leq k_d \leq 1$

I_d este intensitatea luminii reflectate difuz de suprafata

- Dacă i este mai mare ca $\pi/2$, suprafața nu primește lumină de la sursă (sursa de lumină se află în spatele suprafeței).

Modelarea reflexiei luminii ambiante

Lumina ambianta

- Modeleaza lumina provenita de la celelalte obiecte ale scenei 3D: este o sursa de lumina distribuita uniform in spatiu.
- Lumina ambianta este reflectata la fel in orice punct al unei suprafete. Intensitatea sa depinde de intensitatea luminii ambiante si de proprietatile suprafetei.
- Reflexia luminii ambiante este modelata prin:
 $I_a * k_a$, unde
 I_a – este intensitatea luminii ambiante iar
 $0 \leq k_a \leq 1$ este coeficientul de difuzie a luminii ambiante, dependent de materialul suprafetei.

Atunci:

I_d – intensitatea luminii reflectate difuz intr-un punct al unei suprafete

$I_d = \text{reflexia luminii ambiante} + \text{reflexia difuza a luminii de la sursa} = I_a * k_a + I_{\text{sursa}} * k_d * \cos(i)$

$$0 \leq i \leq \pi/2$$

Modelarea atenuării luminii de la o sursa

Sursa direțională: sursa aflată la distanța foarte mare (infinit) de scena 3D

- Vectorul L este același în orice punct al suprafeței
- Pentru 2 suprafețe paralele, cu aceleași proprietăți de material, rezultă aceeași $I_d \rightarrow$ dacă proiecțiile suprafețelor se suprapun în imagine, ele nu se vor distinge.

❖ **Intensitatea luminii descrește proporțional cu inversul pătratului distanței de la sursa de lumină la suprafața.**

$I_d = I_a * k_a + \text{fat} * I_{\text{sursa}} * k_d * \cos(i)$, unde **$\text{fat} = 1/d^2$ este funcția de atenuare a luminii de la o sursa**
 d este distanța de la sursă la punctul de pe suprafață considerat.

- Corecția nu satisface cazurile în care sursa este foarte îndepărtată: atenuare prea mare
- Dacă sursa este la distanță foarte mică de scenă, atenuările obținute pentru două suprafețe aflate la distanțe puțin diferite de sursa, vor fi mult diferite.
- În practică se folosește factorul de atenuare: **$\text{fat} = \min(1/(c_1 + c_2*d + c_3*d^2), 1)$**
 c_1, c_2 și c_3 sunt trei constante care se asociază sursei de lumină.
- Constanta c_1 se alege astfel încât numitorul să nu devină prea mic atunci când sursa este foarte apropiată. Valoarea funcției este limitată la 1 pentru a se asigura atenuarea.

Culoarea luminii reflectate difuz (1)

- Lumina incidenta poate contine mai multe lungimi de unda: → reflectate in mod diferit de o suprafata

$C_{\text{surse}} = [I_{\text{surseR}}, I_{\text{surseG}}, I_{\text{surseB}}]$ – culoarea luminii sursei

$C_a = [I_aR, I_aG, I_aB]$ – culoarea luminii ambientale

- O suprafata reflecta in mod diferit componentele R, G, B ale luminii incidente.
- ❖ Ex: o suprafata care absoarbe din lumina incidenta componentele de verde (G) si albastru (B):
 - Daca lumina incidenta este alba, suprafata va fi vazuta ca rosie:
 $[1,1,1] - [0,1,1] = [1,0,0]$
 - Daca lumina incidenta este rosie, suprafata va fi vazuta tot ca rosie:
 $[1,0,0] - [0,1,1] = [1,0,0]$ (componentele G si B au valoarea zero – nu pot fi extrase)
 - Daca lumina incidenta este verde, suprafata va fi vazuta ca neagra (componenta verde a fost absorbita): $[0,1,0] - [0,1,1] = [0,0,0]$
- Rezulta: difuzia luminii de catre o suprafata depinde de componentele luminii incidente si de proprietatile suprafetei.

Culoarea luminii reflectate difuz (2)

$k_a = [k_{aR}, k_{aG}, k_{aB}]$ – coeficientii de difuzie a componentelor R, G, B din lumina ambientala

$k_d = [k_{dR}, k_{dG}, k_{dB}]$ – coeficientii de difuzie a componentelor R, G, B din lumina provenita de la sursa

- Expresia luminii reflectate difuz se evalueaza separat pentru cele 3 componente ale luminii incidente, R,G,B:

$$I_{d\lambda} = I_{a\lambda} * k_{a\lambda} + f_{at} * I_{sursa\lambda} * k_{d\lambda} * \cos(i), \text{ unde } \lambda: \text{lungimea de unda}$$

sau:

- ❖ $C_d = [I_{dR}, I_{dG}, I_{dB}]$ - culoarea luminii reflectate difuz

$$C_d = [I_{aR} * k_{aR} + f_{at} * I_{sursaR} * k_{dR} * \cos(i), I_{aG} * k_{aG} + f_{at} * I_{sursaG} * k_{dG} * \cos(i), \\ I_{aB} * k_{aB} + f_{at} * I_{sursaB} * k_{dB} * \cos(i)]$$

- ❖ k_a (k_{aR}, k_{aG}, k_{aB}) si k_d (k_{dR}, k_{dG}, k_{dB}) sunt caracteristici ale materialului suprafetei, numite: culoarea ambientala si culoarea difuza a materialului.

Modelarea reflexiei difuze

- Stiind că

$$\cos(i) = \frac{L \cdot N}{(|L| \cdot |N|)} = L_u \cdot N_u$$

unde: N este normala la suprafata in P

L este vectorul din P catre sursa de lumina

$(L \cdot N)$ – produsul scalar al vectorilor L, N

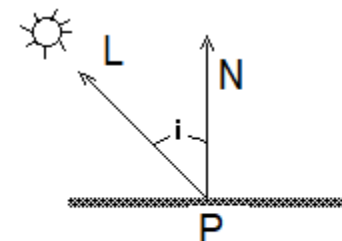
- Rezulta ecuatia care modeleaza lumina reflectata difuz:

$$\diamond Id_\lambda = I_{a\lambda} * k_{a\lambda} + f_{at} * I_{sursa\lambda} * k_{d\lambda} * (L_u \cdot N_u)$$

$C_d = [Id_R, Id_G, Id_B]$ – culoarea luminii reflectate difuz

Pentru a include si cazul in care $i > \pi/2$ (lumina de la sursa nu ajunge in punctul considerat):

$$Id_\lambda = I_{a\lambda} * k_{a\lambda} + f_{at} * I_{sursa\lambda} * k_{d\lambda} * \max((L_u \cdot N_u), 0)$$



Reflexia difuza in punctul P

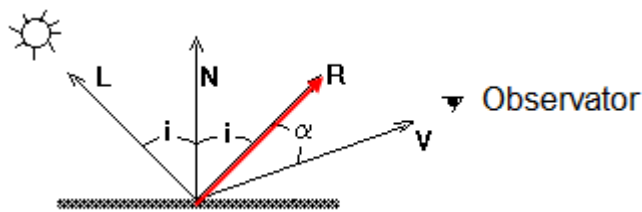
Modele de reflexie -2

Prof. univ. dr. ing. Florica Moldoveanu

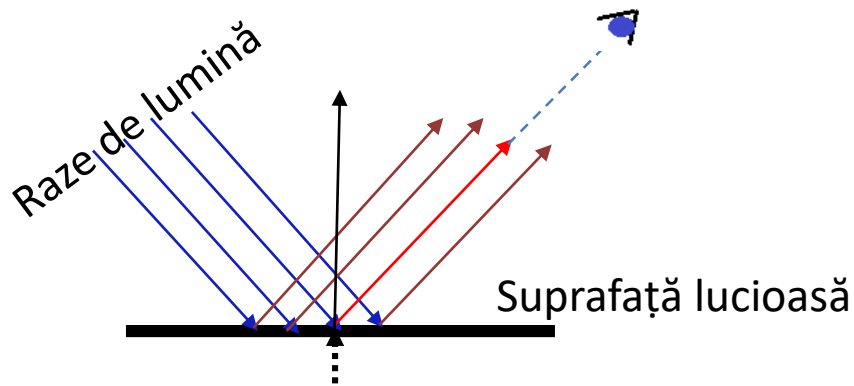
Curs Elemente de Grafică pe Calculator – UPB, Automatică și Calculatoare
2019-2020

Modelarea reflexiei speculare (1)

- Un **reflector perfect**, de exemplu o oglindă, reflectă lumina numai într-o singură direcție, R , care este simetrică cu L față de normala la suprafață $\rightarrow$ numai un observator situat exact pe direcția respectivă va percepe raza reflectată specular:

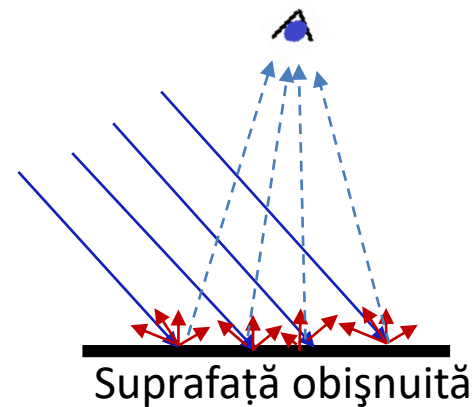


Reflexia speculară



Observatorul vede reflexia speculara a luminii dintr-un singur punct al suprafeței.

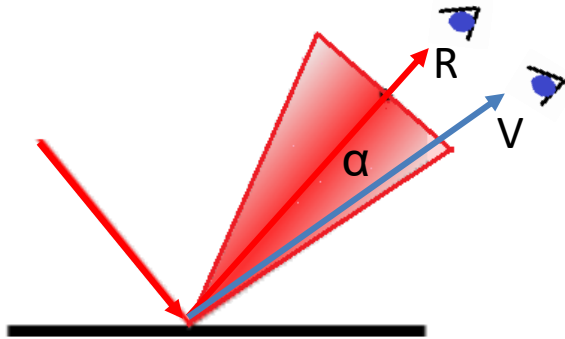
Reflexia difuză



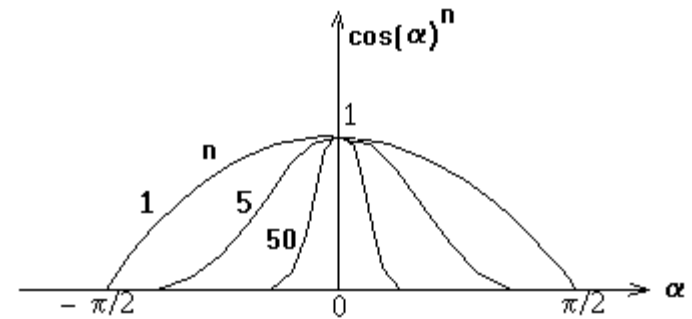
Observatorul vede reflexia difuza a luminii din toate punctele suprafeței. (neregularitățile microscopice ale suprafeței deviază direcția reflexiei în punctele vecine)

Modelarea reflexiei speculare (2)

- In practica se considera ca reflexia speculara se produce nu ca o raza perfecta ci ca un con de lumina, cu o deschidere care depinde de reflectanta suprafatei.



Observatorul aflat in centrul conului vede reflexia maxima



Funcția de distribuție spațială a luminii reflectate specular

- Pentru **materialele imperfect reflectante** cantitatea de lumină care ajunge la observator depinde de **distribuția spațială a luminii reflectate specular**:
 - la suprafețele netede (ex. metale) distribuția este dreaptă și focalizată;
 - la suprafețele cu rugozități (ex. hartia) ea este dispersată.
 - se aproximeaza prin $\cos(\alpha)^n$ (modelul Bui-Tuong Phong) unde **n este exponentul de stralucire al materialului**.

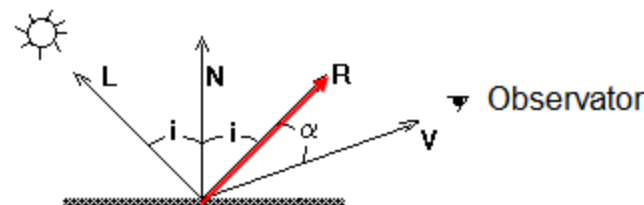
Modelarea reflexiei speculare (3)

Modelul Phong pentru aproximarea reflexiei speculare intr-un punct al unei suprafete 3D:

$$I_{s\lambda} = I_{sursa\lambda} * w(i,\lambda) * \cos(\alpha)^n$$

$w(i, \lambda)$ este funcția de reflectanță,

i - unghiul de incidență iar λ - lungimea de undă a luminii incidente



In practica, $w(i, \lambda)$ este înlocuită cu o constantă determinată experimental, numită **coeficientul de reflexie speculară al materialului, notat $ks\lambda$** .

❖ **Modelul practic al reflexiei speculare:**

$$I_{s\lambda} = I_{sursa\lambda} * ks\lambda * \cos(\alpha)^n$$

$$\cos(\alpha) = \frac{R \cdot V}{(|R| \cdot |V|)} = R_u \cdot V_u$$

Rezulta, adaugand si factorul de atenuare:

$$I_{s\lambda} = I_{sursa\lambda} * fat * ks\lambda * (R_u \cdot V_u)^n$$

Pentru a include si cazul in care $\alpha = 90$:

$$I_{s\lambda} = I_{sursa\lambda} * fat * ks\lambda * \max((R_u \cdot V_u)^n, 0)$$

Modelarea reflexiei speculare (4)

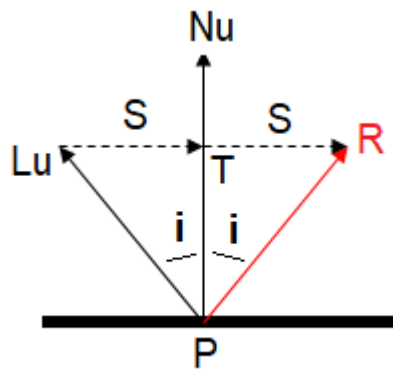
Reflexia speculara nu poate avea loc daca in punctul considerat nu se primeste lumina de la sursa:

$$I_{s\lambda} = I_{um} * I_{sursa\lambda} * f_{at} * k_{s\lambda} * \max((R_u \cdot V_u)^n, 0)$$

$$I_{um} = 1 \text{ daca } (L_u \cdot N_u) > 0 \quad (0 \leq i < \pi/2)$$

$$= 0 \text{ altfel}$$

Calculul directiei reflexiei speculare



- Vectorul R este simetricul vectorului L față de N.
- Proiectia scalară a vectorului Lu pe direcția normalei este mărimea segmentului PT: $|Lu| \cos(i) = \cos(i) = Lu \cdot Nu$
- Proiectia vectorială a lui Lu pe direcția normalei este **vectorul PT**, care are lungimea segmentului PT și direcția normalei:

$$(Lu \cdot Nu) * Nu / |Nu| = (Lu \cdot Nu) * Nu$$

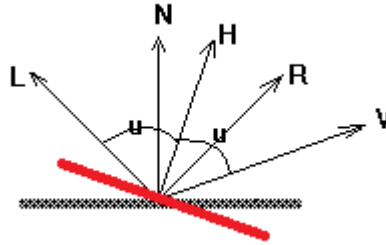
$$R = Lu + 2 * S, \quad PT = Lu + S \rightarrow S = PT - Lu$$

$$R = Lu + 2(PT - Lu) = Lu + 2 * (Lu \cdot Nu) * Nu - 2 * Lu = 2 * (Lu \cdot Nu) * Nu - Lu$$

$$R = 2 * (Lu \cdot Nu) * Nu - Lu$$

Modelarea reflexiei speculare (5)

- O altă formulare a modelului Phong pentru reflexia speculară folosește **vectorul median**, notat cu **H** în figura. El face unghiuri egale cu **L** și cu **V**.



- ❖ Justificare: dacă suprafața ar fi orientată astfel încât normala să aibă direcția lui **H**, atunci observatorul ar percepe lumina speculară maximă (**V** aliniat cu **R**) → unghiul dintre **L** și **H** determină intensitatea reflexiei speculare.

- ❖ Termenul care exprimă reflexia speculară este în acest caz:

$$I_{s\lambda} = I_{sursa\lambda} * f_{at} * k_{s\lambda} * (N_u \cdot H_u)^n, \quad \text{unde } H_u = (L_u + V_u) / |L_u + V_u|$$

Atunci când sursa de lumină și observatorul sunt la infinit (**L** și **V** aceleași în toate punctele unui poligon), H_u este pe suprafața poligonului.

Modelul de iluminare locala(1)

- lumina reflectata într-un punct al unei suprafete -

Modelul de iluminare locala (numit și Modelul de reflexie Phong)

Combina culorile datorate:

reflexiei luminii ambientale – culoarea_ambientala,

reflexiei difuze a luminii de la o sursa / surse – culoarea_difuza

reflexiei speculare a luminii de la o sursa/ surse – culoarea_speculara

❖ Reflexia luminii într-un punct al unei suprafețe determina culoarea sa în acel punct.

$$I_{\lambda} = I_{a\lambda} * k_{a\lambda} + f_{at} * I_{sursa\lambda} [k_{d\lambda} * \max((L_u \cdot N_u), 0) + I_{um} * k_{s\lambda} * \max((R_u \cdot V_u)^n, 0)]$$

$$I_{um} = 1 \text{ daca } (L_u \cdot N_u) > 0 \quad (0 \leq i < \pi/2) \\ = 0 \text{ altfel}$$

Modelul de iluminare locala(2)

- ❖ Dacă scena 3D este luminată de m surse de lumină:

$$I_{\lambda} = I_{a\lambda} * k_{a\lambda} + \sum_{1 \leq i \leq m} f_{at_i} * I_{sursa\lambda_i} * [k_{d\lambda} * \max((L_{u_i} \cdot N_u), 0) + I_{um_i} * k_{s\lambda} * \max((R_{u_i} \cdot V_u)^n, 0)]$$

- ❖ La cele 3 componente se poate adauga culoarea_emisa de suprafata. Rezulta:

Culoarea intr-un punct al unei suprafete

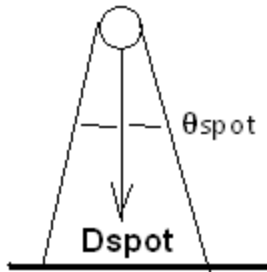
= culoarea_emisa + culoarea_ambientala + culoarea_difuza + culoarea_speculara

- ❖ **Culoarea emisa :**

- lumina emisa de o suprafata
- independenta de sursele de lumina
- aceeasi in orice punct al suprafetei
- nu lumineaza obiectele din jur
- culoarea_emisa = K_e , o constanta

Efectul de spot

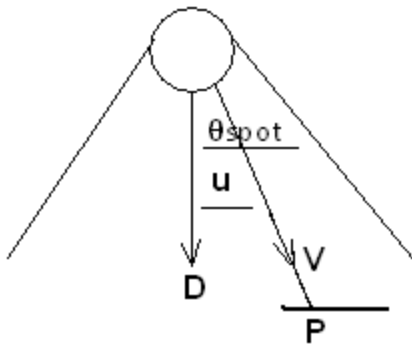
- In modelul de iluminare locala, sursa de lumina este considerata punctiforma. Lumina produsa de o sursa punctiforma este omnidirectionala.
- Efectul de spot simuleaza o sursa de lumina care produce un con de lumina.
- Numai suprafetele aflate in conul de lumina primesc lumina de la sursa.



Conul de imprastiere a luminii este definit prin:

- Pozitia spotului (sursei de lumina)
- Directia spotului
- Imprastierea luminii spotului (θ_{spot})

Un punct P al unei suprafete primeste lumina de la spot
daca $u < \theta_{spot}$:



sau

$$\cos(u) > \cos(\theta_{spot})$$

$$Vu \cdot Du > \cos(\theta_{spot})$$

Modele de “shading”

Prof. univ. dr. ing. Florica Moldoveanu

Curs Elemente de Grafică pe Calculator – UPB, Automatică și Calculatoare
2019-2020

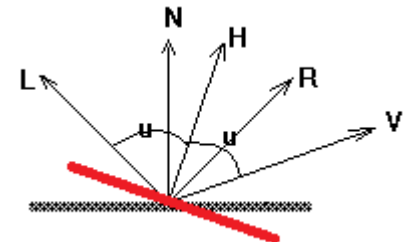
Calculul culorii fragmentelor la redarea suprafețelor 3D (Shading models) (1)

- În vederea vizualizării, suprafețele 3D sunt descompuse în fațete poligonale (triunghiulare) conectate. Poligoanele procesate în banda grafică pot fi fețele unui corp poliedral (fețe plate) sau pot proveni din descompunerea unei suprafețe oarecare (fețele sunt petice de suprafața curba, approximate prin fațete plane).
- La momentul rasterizării nu se cunoaște din ce suprafață face parte un poligon.

Modelul LAMBERT(1) (Iluminare constantă sau plată)

- Toate fragmentele unui poligon sunt afișate cu aceeași culoare, calculată folosind modelul de iluminare locală, de regulă considerând numai componenta difuză:

$$I_d \lambda = I_a \lambda * k_a \lambda + f_a t * I_{sursa \lambda} * [k_d \lambda * \max((N_u \cdot L_u), 0) + I_{um} * \max((N_u \cdot H_u)^n, 0)]$$



Modelul LAMBERT(2)

Modelul se bazează pe următoarele presupuneri:

1. Poligonul face parte din suprafața de vizualizat și nu este o aproximare a unui petic de suprafață curbă: **vectorul Nu este același în orice punct al poligonului.**
 2. Sursa de lumină este la infinit: produsul scalar ($Nu \bullet Lu$) este atunci constant pe întreaga suprafață a poligonului.
 3. Observatorul este la infinit: ($Nu \bullet Hu$) este constant pe suprafața poligonului;
- Dacă ultimele două cerințe nu sunt satisfăcute, se poate adopta o convenție de calcul al vectorilor L și V pentru un întreg poligon. De exemplu, cei doi vectori pot fi calculați în centrul poligonului.
 - Dacă prima cerință nu este îndeplinită, aproximarea unei suprafețe curbe prin rețeaua de fațete poligonale este vizibilă deoarece intensitățile calculate pentru fațete adiacente cu orientare diferită sunt diferite.

Efectul de banda Mach

- Efectul de banda Mach (descoperit de Mach în 1865) este o iluzie optica datorata filtrarii componentei de luminozitate a imaginii formate pe retina, de catre sistemul vizual uman.
 - Efectul mărește contrastul dintre zone adiacente cu intensitate usor diferita, la frontiera dintre zone, declanșând detecția frontierei in sistemul vizual uman.
 - Atunci cand o suprafața curba (de exemplu o sfera) este descompusa in fațete poligonale, este de dorit ca laturile fațetelor sa nu fie vizibile in imaginea suprafetei.
 - In cazul folosirii modelului Lambert, culorile fațetelor adiacente cu orientari usor diferite vor fi usor diferite, iar efectul de banda Mach va mări percepția schimbării de culoare pe laturile fațetelor adiacente.
- In concluzie, modelul Lambert este adecvat vizualizarii obiectelor poliedrale.

Modelul LAMBERT(3)

Indicatii de implementare

//vertex shader

```
layout(location = 0) in vec3 v_position;
```

```
layout(location = 1) in vec3 v_normal;
```

```
uniform vec3 light; // directia sursei de lumina
```

```
uniform vec3 eye; // directia observatorului
```

```
flat out vec3 color; // culoarea varfului "provocator" va fi folosita la afisarea
```

```
    // fragmentelor; in mod implicit ultimul varf al primitivei (ex.: al 3-lea varf pt un triunghi)
```

```
void main() {
```

```
    // calculeaza light_dir, eye_dir și culoarea, în pozitia varfului, aplicand modelul de iluminare
```

```
    .....
```

```
    color = ambient + diffuse + specular;
```

```
}
```

//fragment shader

```
flat in vec3 color; // culoarea varfului "provocator"
```

```
void main() {
```

```
    gl_FragColor = color;
```

```
}
```


Modelul Gouraud (1)

- ❑ La momentul rasterizării unui poligon, fiecărui vârf al poligonului îi este asociată o culoare.
- ❑ Procesorul de rasterizare calculează culoarea fiecărui fragment al poligonului astfel:
 - prin interpolare între culorile varfurilor, pentru fragmentele de pe laturi
 - prin interpolare între culorile din capetele segmentului interior pe care se afla fragmentul.

Implementare

1. In **Vertex shader se calculează culoarea varfului**, aplicând modelul de iluminare locală în poziția varfului (cu N , L , V în coordonate globale, primite din programul de aplicație)

Culoarea varfului este ieșire din vertex shader: **out vec3 color**

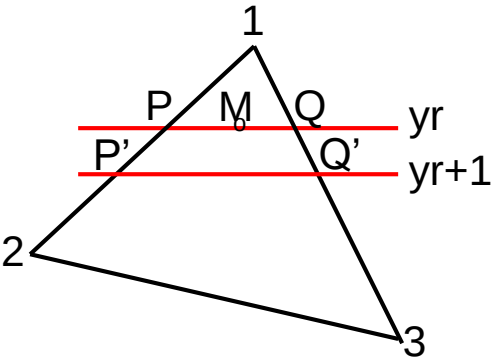
2. **Fragment shader primește ca intrare culoarea fragmentului calculată de procesorul de rasterizare (in vec3 color).**

❖ Normala unui vârf, necesară în vertex shader:

- poate fi calculată exact, folosind ecuația analitică a suprafeței pe care se afla varful
- poate fi aproximată ca medie aritmetică a normalelor la fețele adiacente în varf

Modelul Gouraud (2)

Interpolarea liniara a culorilor



$$y = y_1 + t (y_2 - y_1) \rightarrow t_P = (y_r - y_1) / (y_2 - y_1)$$

$$I_P = I_1 + t_P (I_2 - I_1) = I_1 + (I_2 - I_1) (y_r - y_1) / (y_2 - y_1) = I_1 + (y_r - y_1) * C_{1-2}$$

$$C_{1-2} = (I_2 - I_1) / (y_2 - y_1) - \text{constanta pentru latura 1-2}$$

$$I_Q = I_1 + (I_3 - I_1) (y_r - y_1) / (y_3 - y_1) = I_1 + (y_r - y_1) * C_{1-3}$$

$$C_{1-3} = (I_3 - I_1) / (y_3 - y_1) - \text{constanta pentru latura 1-3}$$

$$x = x_P + t(x_Q - x_P) \rightarrow t_M = (x_M - x_P) / (x_Q - x_P)$$

$$I_M = I_P + t_M (I_Q - I_P) = I_P + (I_Q - I_P) (x_M - x_P) / (x_Q - x_P) = I_P + (x_M - x_P) * C_{P-Q}$$

$$C_{P-Q} = (I_Q - I_P) / (x_Q - x_P) - \text{constanta pentru segmentul P-Q}$$

Calculul incremental al culorilor

$$I_{P'} = I_1 + (I_2 - I_1) (y_{r+1} - y_1) / (y_2 - y_1) = I_1 + (y_{r+1} - y_1) * C_{1-2} = I_P + C_{1-2}$$

$$I_{P'} = I_P + C_{1-2} - \text{analog, } I_{Q'} = I_Q + C_{1-3}$$

$$M' (x_M + 1, y_s)$$

$$I_{M'} = I_P + (x_M + 1 - x_P) * C_{P-Q} \rightarrow I_{M'} = I_M + C_{P-Q}$$

Modelul Gouraud (3)

Aprecieri asupra modelului Gouraud

- Calculul culorilor fragmentelor este integrat în algoritmul de rasterizare a poligoanelor
- Calcule simple pentru culoarea unui fragment (calcul incremental) → rapid
- Nu permite calculul luminii reflectate specular pentru fragmentele interioare unui poligon (intensitatile fragmentelor interioare nu pot fi mai mari decat cele din varfuri)
- Reduce efectul de banda Mach, dar nu il elimina:
 - Culorile pe latura de adiacenta a 2 poligoane sunt aceleasi
 - Nu tine cont de tangentele la fetele adiacente pe o latura (culorile de pe laturi se calculeaza folosind numai culorile din varfuri)
 - Efectul de banda Mach poate fi observat in vecinatatea siluetei unei suprafete 3D si a zonelor de curbura mare.

Modelul Phong (1)

- ❑ La momentul rasterizării unui poligon, fiecărui vârf îi este asociată normala vârfului.
- ❑ Procesorul de rasterizare calculează normala fiecărui fragment al poligonului astfel:
 - prin interpolare între normalele varfurilor, pentru fragmentele de pe laturi;
 - prin interpolare între normalele capetelor segmentului interior pe care se afla fragmentul (la fel ca în modelul Gouraud, pentru culori).

Implementare

- Normala vârfului este transmisă din programul de aplicație, ca și pentru modelul Gouraud.
- Normala și poziția vârfului, în coordonate globale, sunt ieșiri din vertex shader.
- Procesorul de rasterizare interpolează liniar pozițiile și normalele varfurilor, transmitând la fragment shader poziția și normala fragmentului.
- ❑ În fragment shader se calculează culoarea fragmentului aplicând modelul de iluminare locală în poziția fragmentului.

Modelul Phong (2)

Dezavantaje fata de modelul Gouraud

- ❑ Componentele N_x , N_y , N_z ale normalei unui fragment se obțin (in algoritmul de rasterizare) printr-un calcul incremental analog cu cel folosit pentru calculul culorilor in modelul Gouraud dar, pentru folosirea în calculul culorii, **normala trebuie să fie normalizată în fragment shader**:

$$N_u = N/|N|, \text{ unde } |N| = (N_x^2 + N_y^2 + N_z^2)^{0.5} \quad (\text{se foloseste o functie GLSL})$$

- ❑ De asemenea, vectorii L (catre sursa de lumina) si V (catre observator), folositi in modelul de iluminare locala la nivel de fragment, sunt calculati si normalizati in programul fragment shader pentru a fi folositi in calculul culorii fragmentului.

➤ **Calcule mai complexe la nivel de fragment decât in modelul Gouraud**

Avantaje

- ❑ **Permite redarea reflexiei speculare in orice fragment al unui poligon**
- ❑ **Reduce mult efectul de banda Mach.**

Transformarea de vizualizare din spațiul 3D

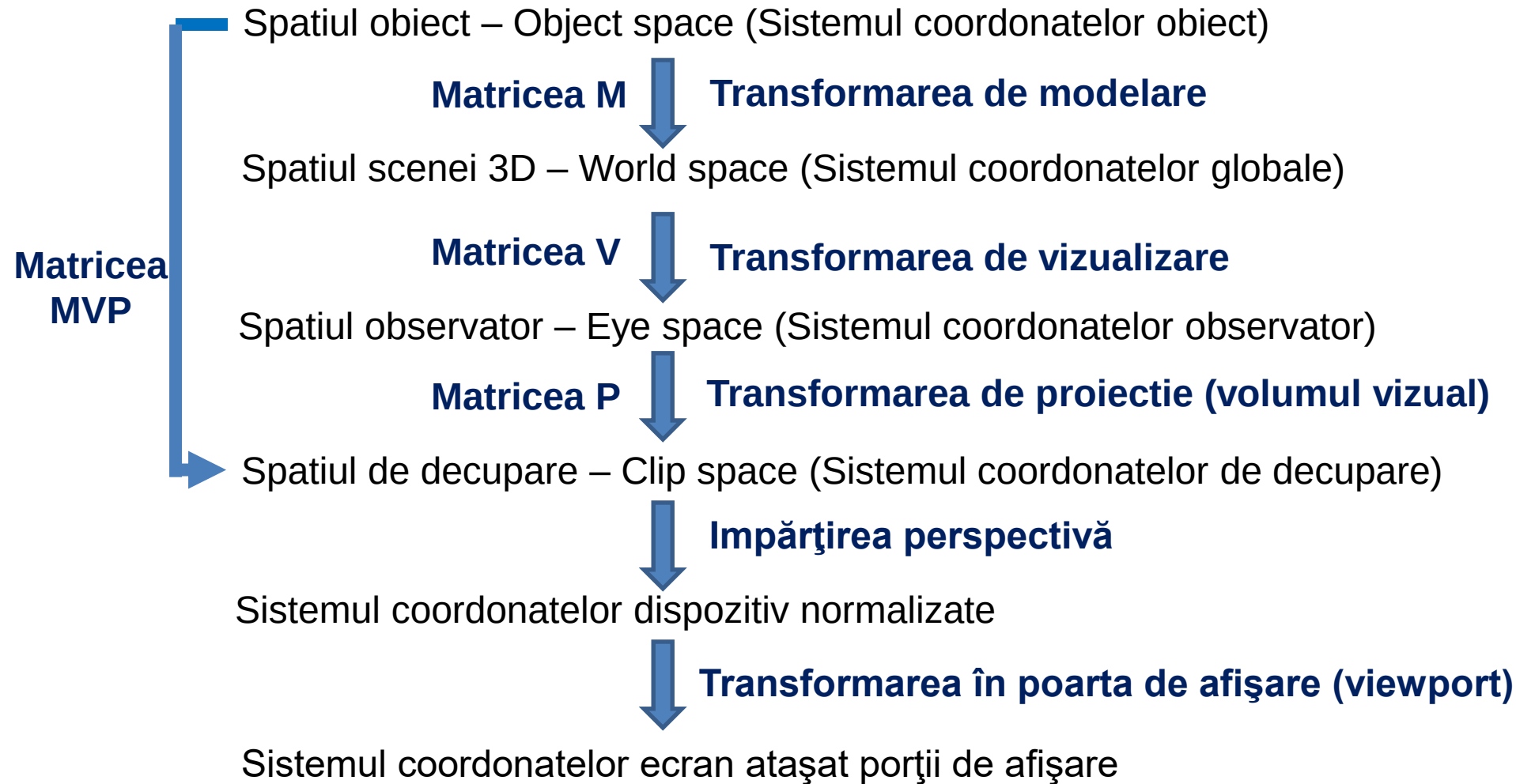
Prof. univ. dr. ing. Florica Moldoveanu

Curs Elemente de Grafică pe Calculator – UPB, Automatică și Calculatoare
2019-2020

Transformarea de vizualizare din spațiul 3D

- Are ca scop **obținerea unei vederi a unei scene 3D** pe o suprafață de afișare.
- Poate fi comparată cu transformarea efectuată de o cameră foto asupra lumii reale pentru a se obține imaginea dintr-o fotografie.
 - Imaginea obținută depinde de:
 - poziția camerei (observatorului)
 - orientarea camerei
 - ceea ce vede ochiul prin vizorul camerei foto (volumul vizual)
- Este o transformare compusă din mai multe transformări, între care și o proiecție $R^3 \rightarrow R^2$
- Transformările se efectuează asupra vârfurilor obiectelor din scena 3D.
- Include și operația de decupare (clipping) a primitivelor grafice la marginea volumului vizual (eliminarea părților din scena 3D care sunt în afara volumului vizual al vederii curente).

Transformarea varfurilor în OpenGL



Matricile transformării varfurilor obiectelor 3D

M – matricea de modelare: difera de la obiect la obiect

V – matricea de vizualizare: aceeași pentru întreaga scenă vizualizată

P – matricea de proiecție: aceeași pentru întreaga scenă vizualizată

MV = V · M (matricea “model-view”)

MVP = P · V · M (matricea “model-view-projection”)

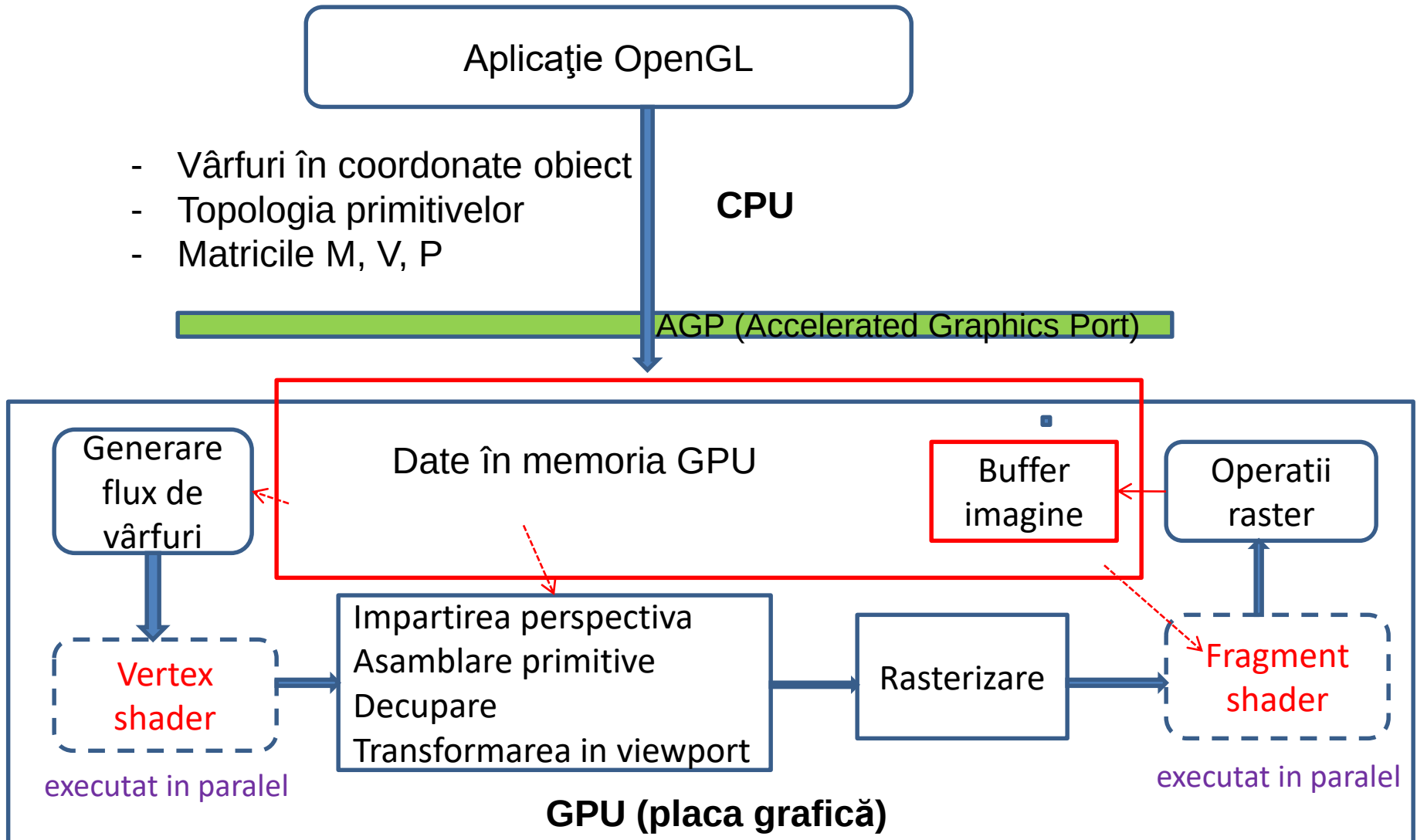
Transformarea vârfurilor din coordonate obiect în coordonate de decupare (clip):

$$[x_c, y_c, z_c]^T = PVM \cdot [x_o, y_o, z_o]^T$$

- Matricile sunt calculate în programul OpenGL folosind funcții ale bibliotecii **glm** și transferate la GPU
- Transformarea vârfurilor este efectuată în programul **Vertex Shader executat pe GPU**, folosindu-se matricea MVP.
- Urmează: împărțirea perspectivă prin care se obțin coord. carteziane ale varfurilor; asamblarea primitivelor folosind topologia transmisă la GPU de programul OpenGL; decuparea primitivelor la frontiera volumului vizual canonic; transformarea primitivelor decupate, în viewport.

Banda grafică- a V-a generație GPU

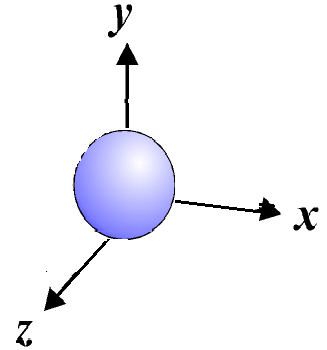
➤ Incepend cu OpenGL 3.0:



Spatiul obiect

Spatiul obiect (Sistemul coordonatelor obiect)

- In mod uzual, fiecare obiect este definit in propriul sau sistem de coordonate (sistemul coordonatelor locale – coordonate obiect)
 - Sistem de coordonate carteziane 3D « dreapta »
 - Exemple: cubul, sfera, scheletul unui personaj animat, elemente de mobilier, etc.
- De regula, obiectul este definit centrat in originea sistemului de coordonate.
- Avantaje: usurinta modelarii și reutilizarea modelelor obiectelor 3D

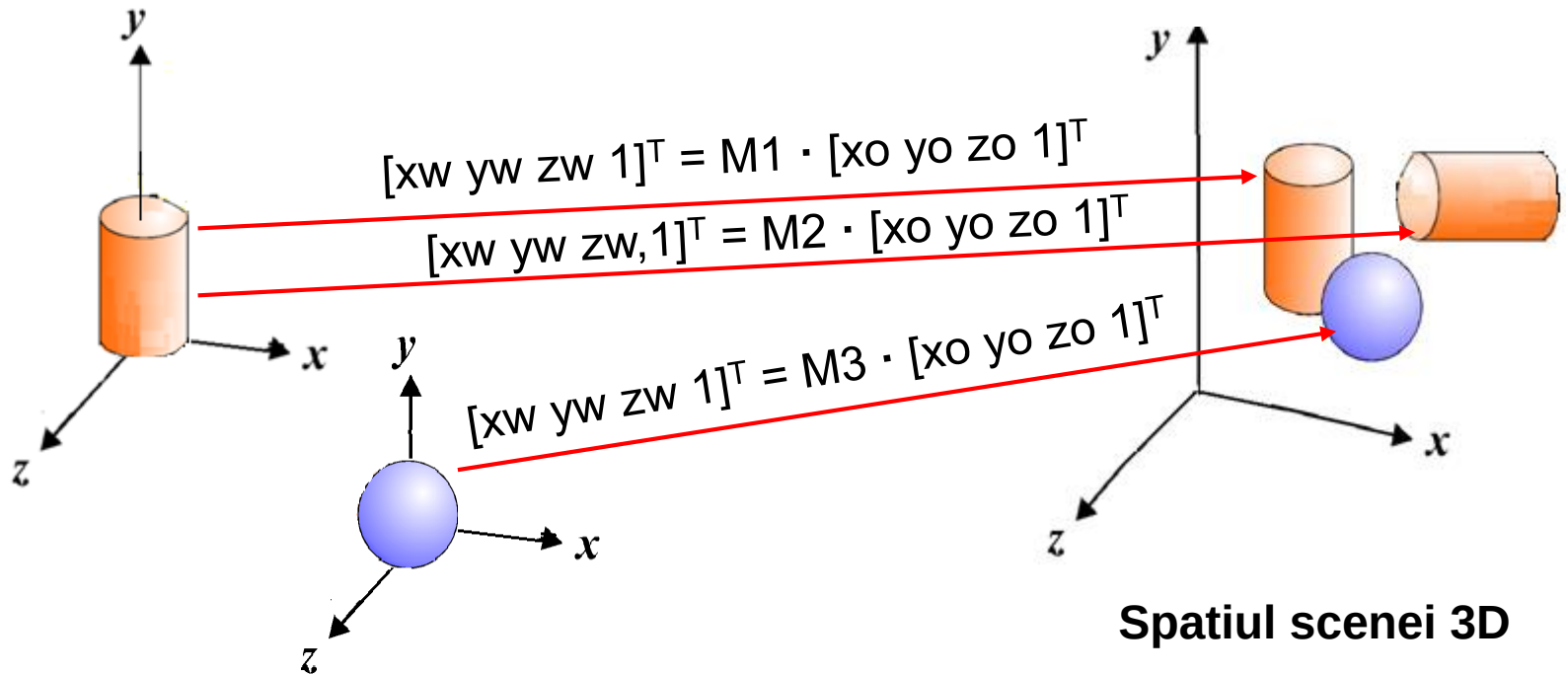


Spatiul scenei 3D -World space (Sistemul coordonatelor globale)

- Spațiul in care este compusă scena 3D
- Scena - raportată la un sistem de referință global – sistem de coordonate carteziane 3D “dreapta”, numit *Sistemul coordonatelor globale*
- Originea sistemului – aleasa de modelatorul scenei

Spatiul scenei 3D

Transformarea de modelare



$[x_o \ y_o \ z_o \ 1]$: coord omogene varf în sist de coord al unui obiect

$[x_w \ y_w \ z_w \ 1]$: coord omogene varf în sist de coord globale

Transformarea de modelare:



Transformare geometrica compusa. Exemple:

$$M1 = T(tx1, ty1, tz1)$$

$$M2 = T(tx2, ty2, tz2) * R_{oz} (1.57)$$

Spatiul scenei 3D

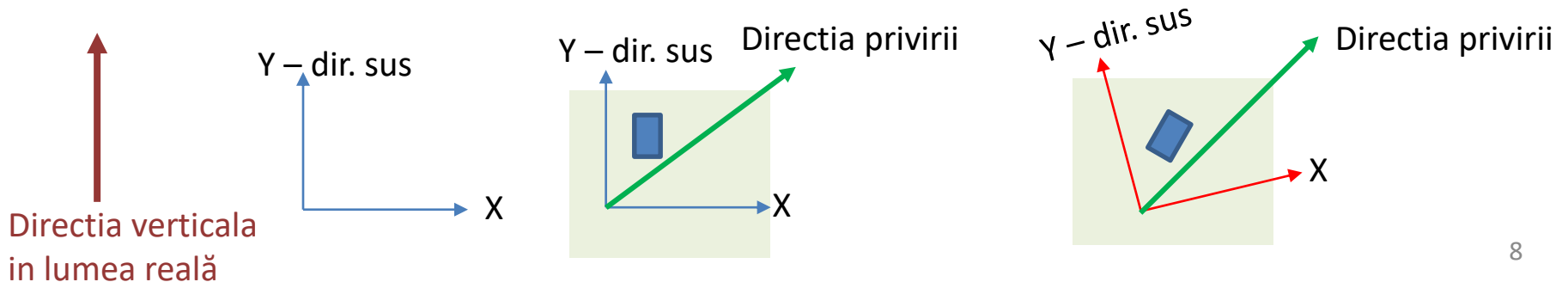
Obiecte definite în sistemul coordonatelor globale

Spatiul camerei (observator)

Analogie cu aparatul de fotografiat (camera reala)

Un aparat foto poate produce o fotografie in care apar o parte din obiectele spatiului real (3D).

- Continutul fotografiei depinde de:
 - pozitia aparatului in spatial real
 - orientarea acestuia: directia in care priveste fotograful
 - rotatia aparatului față de pozitia sa normală
- Fotografia este formata intr-un plan 2D, fiind raportata la un sistem cartezian 2D, XOY:
- **In pozitia normala a aparatului, directia axei OY (axa verticala) este directia “sus” a aparatului.**
- Directia in care priveste fotograful este perpendiculara pe planul imaginii, formand impreuna cu axele OX, OY un sistem de coord carteziene 3D.
- Fotograful poate roti aparatul de fotografiat, astfel incat directia “sus” a aparatului se schimbă.



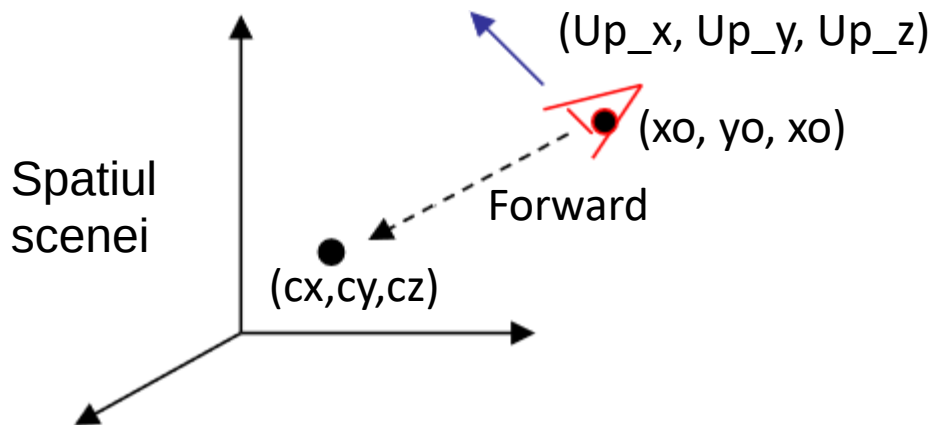
Spațiul camerei (observator)

Sistemul de coordonate observator

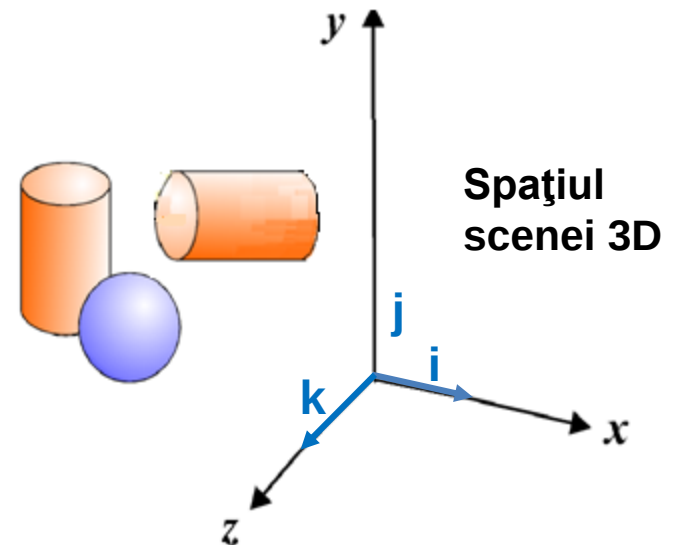
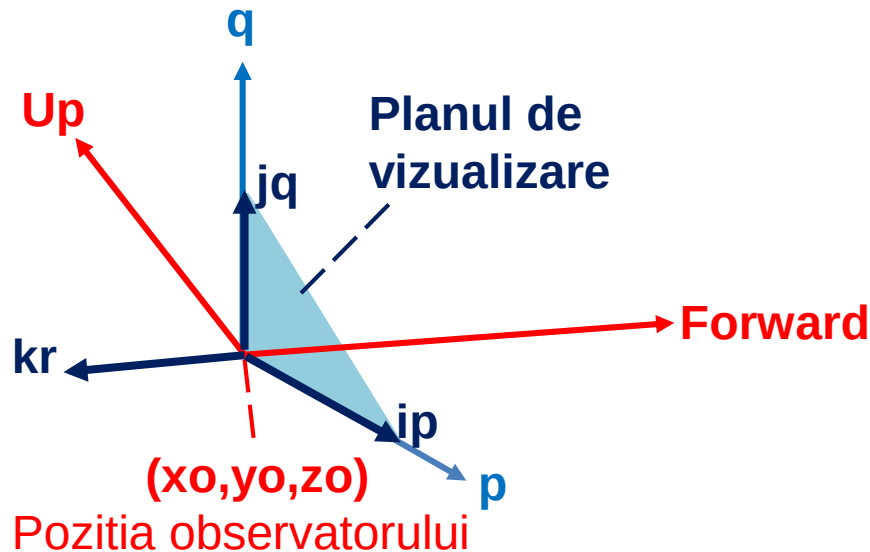
- Spațiul camerei este spațiul scenei 3D văzut din poziția camerei virtuale (observatorului) pe direcția în care privește observatorul.
- Spațiul camerei este raportat la un sistem de coordonate atasat camerei – sistemul de coordonate observator.

➤ **Sistemul de coordonate observator este definit prin 3 parametri:**

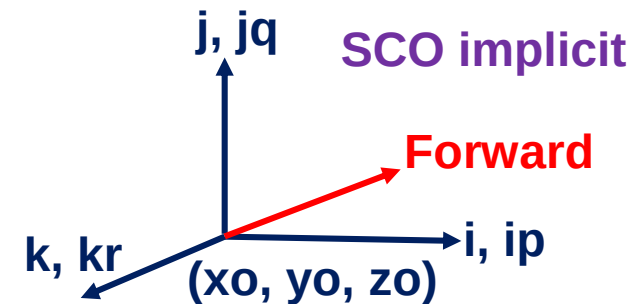
1. Originea sa: **poziția camerei virtuale** în spațiul scenei (x_o, y_o, z_o).
2. **Directia în care privește observatorul** (spre un punct din scenă) – **vectorul Forward**
3. **Directia “sus” a camerei (vectorul Up)**: reprezintă **rotatia camerei**



Sistemul de coordonate observator (SCO)

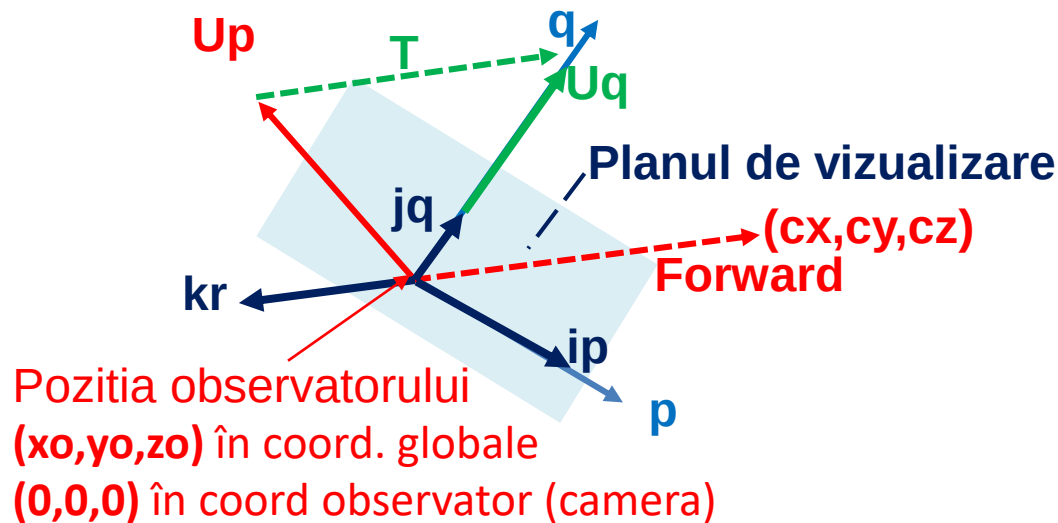


- (x_0, y_0, z_0) : originea sistemului de coordonate observator (pozitia camerei)
- (x_0, y_0, z_0) impreuna cu vectorul **Forward** definesc un plan, numit **planul de vizualizare**.
- **Forward** – normala la planul de vizualizare
- axa q : axa verticală a planului de vizualizare
- axa p : axa orizontală a planului de vizualizare
- (ip, jq, kr) formeaza un sistem de coordonate 3D dreapta



ip, jq, kr : versorii axelor sistemului de coordonate observator coincid cu cei ai sist. coordonatelor globale

Determinarea axelor sistemului de coordonate observator (1)



$kr = - \text{Forward} / |\text{Forward}|$
 Versorul direcției axei z a SCO

- Uq : proiecția ortogonală a vectorului Up în planul de vizualizare: $Uq = Up + T$
- Uq dă direcția axei verticale a planului de vizualizare

$$T \parallel (-kr) \rightarrow T = s \cdot (-kr) = -s \cdot kr \quad (s \text{ un scalar oarecare}) \rightarrow Uq = Up - s \cdot kr$$

$$Uq \text{ perpendicular pe } kr \rightarrow Uq \cdot kr = 0 \rightarrow (Up - s \cdot kr) \cdot kr = 0 \rightarrow Up \cdot kr - s \cdot kr \cdot kr = 0 \rightarrow s = Up \cdot kr$$

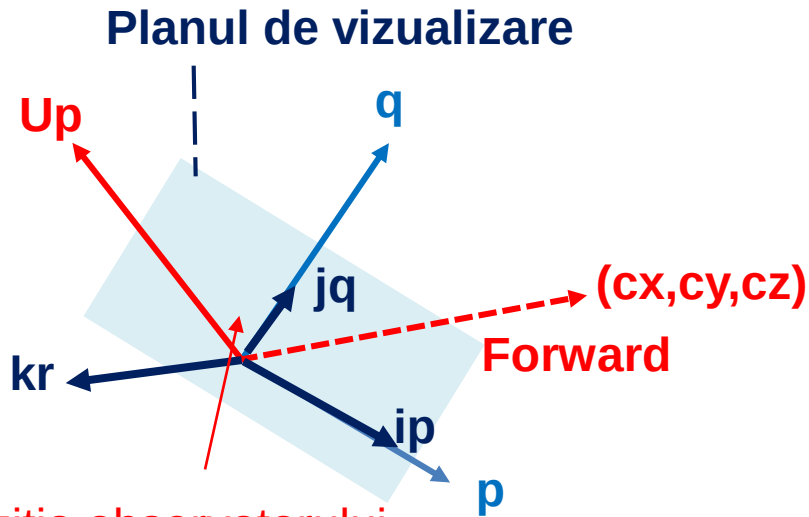
$$Uq = Up - (Up \cdot kr) \cdot kr$$

$jq = Uq / |Uq|$ este versorul axei verticale a planului de vizualizare

$ip = jq \times kr$ este versorul axei orizontale a planului de vizualizare

(ip, jq, kr) formează un sistem de coordonate carteziane 3D dreapta

*Determinarea axelor sistemului de
coordonate observator (2)
- o alta metoda -*



Pozitia observatorului
(x_o, y_o, z_o) în coord. globale
($0, 0, 0$) în coord observator (camera)

$$k_r = - \text{Forward} / |\text{Forward}|$$

Versorul directiei axei z a SCO

p este un vector perpendicular pe
planul format de vectorii U_p si k_r :

$$p = U_p \times k_r$$

$$\mathbf{ip} = \mathbf{p} / |\mathbf{p}|$$

(ip, jq, kr) trebuie sa formeze un sistem de coordonate carteziene 3D dreapta

Deci:

$\mathbf{jq} = \mathbf{kr} \times \mathbf{ip}$ este versorul axei verticale a planului de vizualizare

Matricea transformării de vizualizare (V) -1

Transformarea de vizualizare: din spatiul scenei → în spatiul camerei

$$[x_e \ y_e \ z_e \ 1]^T = V \cdot [x_w \ y_w \ z_w \ 1]^T \quad \text{eye coordinates} \leftarrow \text{world coordinates}$$

Sistemul de coordonate observator implicit in OpenGL

- poziția observatorului: **originea sistemului de coordonate globale**
- direcția în care privește observatorul: **direcția negativă a axei OZ**
- direcția sus a planului de vizualizare: **direcția pozitivă a axei OY**

Cu aceste valori implicite, transformarea de vizualizare este transformarea identica.

Pentru calculul matricei transformarii de vizualizare poate fi folosita functia **lookAt**:

glm::mat4 V =

glm::lookAt (glm::vec3 (xo,yo,zo), glm::vec3 (cx,cy,cz), glm::vec3 (Upx, Upy, Upz))

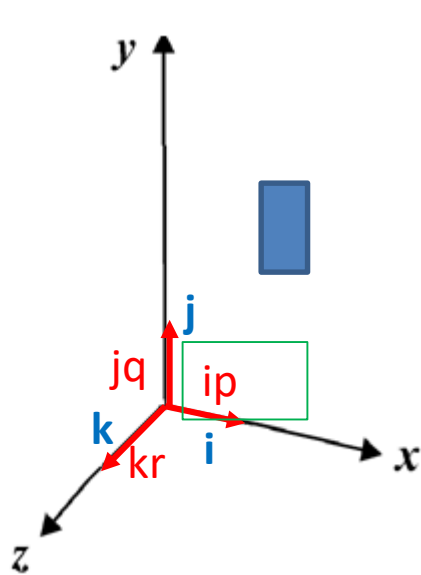
(xo, yo, zo): **poziția observatorului**

(cx,cy,cz): centrul de interes - punctul din scena 3D catre care priveste observatorul;

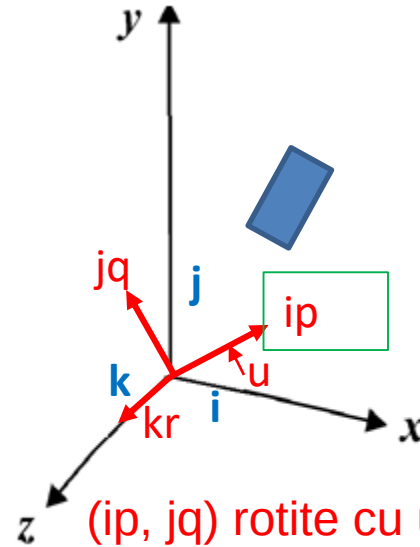
vectorul (xo,yo,zo) → (cx,cy,cz) reprezinta **vectorul Forward**

(Upx, Upy, Upz): **vectorul « sus » al camerei (Up)**

Matricea transformării de vizualizare -2



$V = I$ (matricea identica)



(ip, jq) rotite cu unghiul u in jurul axei oz

$V?$

V : matricea care transforma varfurile obiectelor din spatiul scenei in spatiul camerei

(observator - eye) $\rightarrow [x_e \ y_e \ z_e \ 1]^T = R_{OZ}(-u) \cdot [x_w \ y_w \ z_w \ 1]^T$

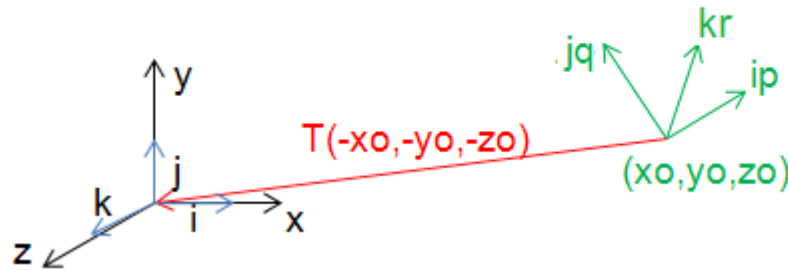
Matricea transformarii de vizualizare: $V = R_{OZ}(-u)$

➤ Aplicand V axelor sistemului de coordonate observator, se vor suprapune pe cele ale sistemului de coordonate globale.

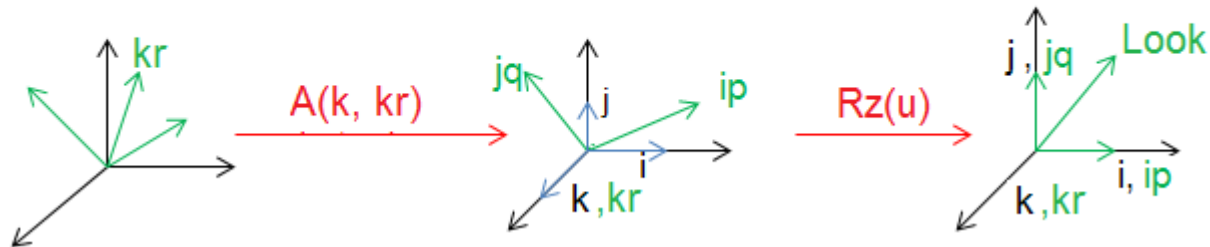
Matricea transformării de vizualizare -3

Sistemul de coordonate al scenei

Sistemul de coordonate observator



Sistemul de coordonate observator in OpenGL



- $A(k, kr)$ este o transformare prin care se aliniaza axele z ale celor 2 sisteme de coordonate: consta din 2 rotatii – o rotatie in jurul axei X si una in jurul axei Y .
- Prin rotatia in jurul axei z (Rz) se aliniaza celelalte 2 axe.

Matricea transformarii de vizualizare: $V = Rz(u) * A(k, kr) * T(-x_o, -y_o, -z_o)$

Transformarea de modelare și vizualizare

din sistemul de coordonate obiect → în sistemul de coordonate observator

❖ Transformare compusa (model-view) : $MV = V \cdot M$

M – matricea de modelare folosita pentru transformarea unui obiect in spatiul scenei

V – matricea de vizualizare a scenei

Transformarea varfurilor din coordonate obiect in coordonate observator (eye):

$$[x_e \ y_e \ z_e \ w_e]^T = V \cdot M \cdot [x_o \ y_o \ z_o \ w_o]^T = MV \cdot [x_o \ y_o \ z_o \ w_o]^T$$

➤ Pentru calculele de iluminare a obiectelor scenei 3D se folosesc vectori normali atasati varfurilor obiectelor.

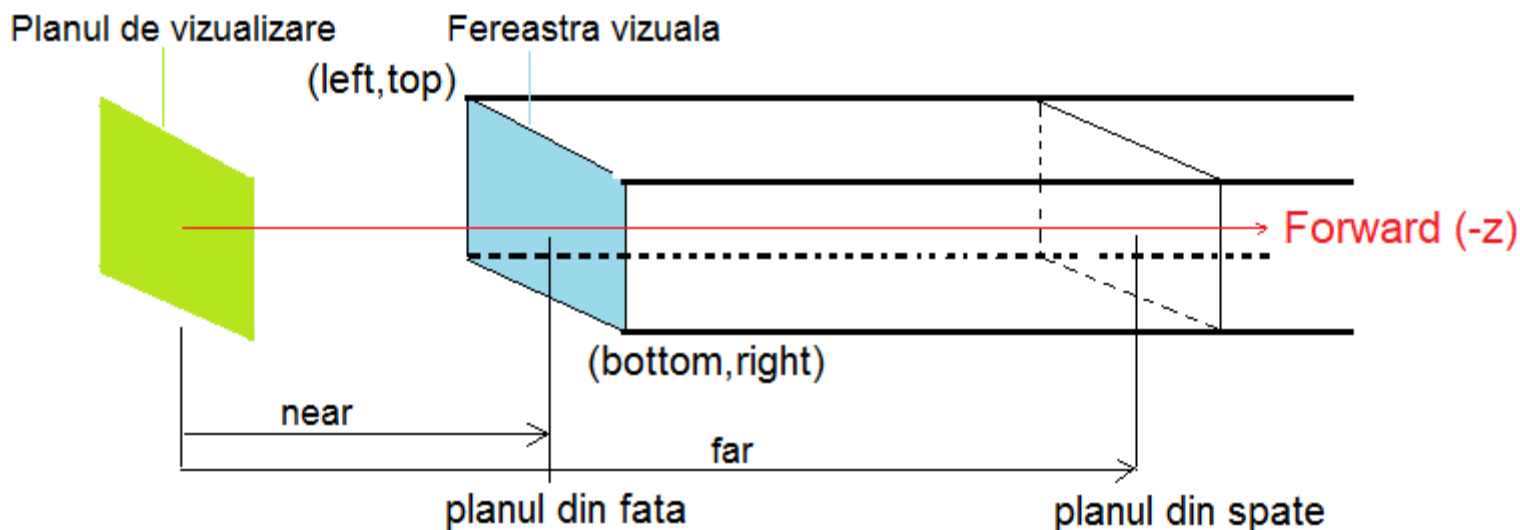
Transformarea normalelor din coordonate obiect in coordonate observator:

$$[n_{x_e} \ n_{y_e} \ n_{z_e} \ w_e]^T = (MV^{-1})^T \cdot [n_{x_o} \ n_{y_o} \ n_{z_o} \ n_{w_o}]^T$$

Transformarea de proiectie (1)

Determinata de **volumul vizual** definit de programator

Volumul vizual al proiectiei ortografice: paralelipiped dreptunghic cu laturile paralele cu axa z a sistemului de coordonate observator (SCO), delimitat in adancime de planul din fata (near) si planul din spate (far) al vederii; near si far sunt distante (pozitive) față de planul de vizualizare masurate pe axa z negativa a SCO.

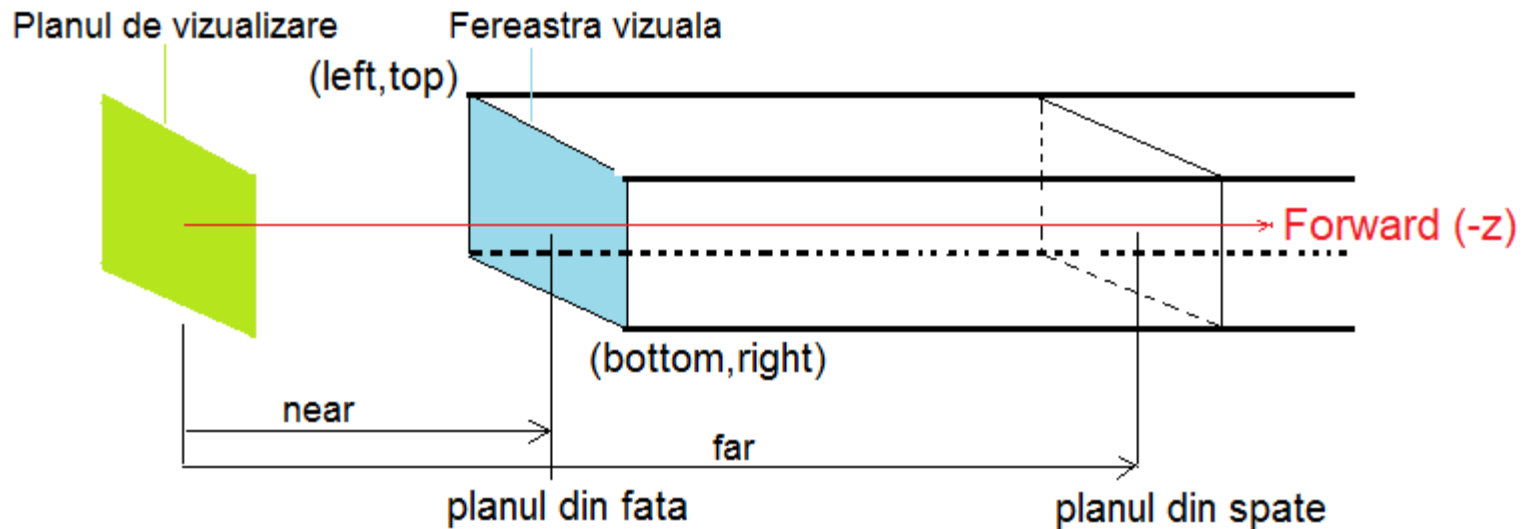


- Coordonatele colturilor ferestrei vizuale sunt raportate la sistemul de coordonate al planului din față, care are axele paralele cu cele ale planului de vizualizare.

Transformarea de proiecție (2)

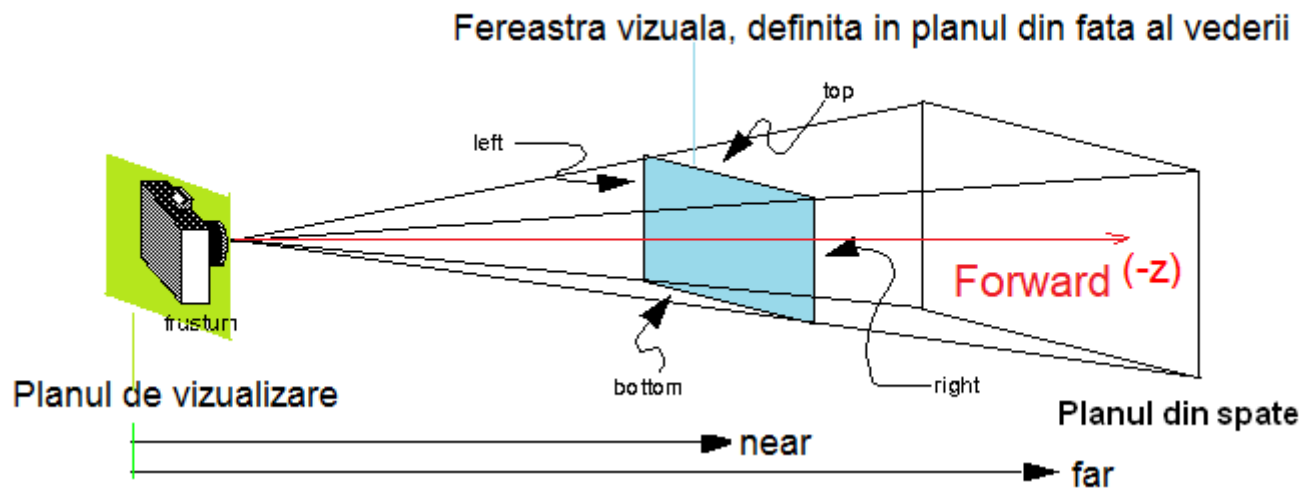
Proiecția ortografică

- Se efectuează proiecția ortografică în fereastra vizuală a obiectelor cuprinse în volumul vizual. Primitivele grafice situate complet în afara volumului vizual sunt eliminate iar cele care intersectează volumul sunt decupate la frontiera volumului creindu-se noi primitive.



Transformarea de proiecție (3)

Volumul vizual al proiecției perspective (View frustum): trunchi de piramidă delimitat în adâncime de planul din față (near) și planul din spate (far) al vederii, cu vârful în poziția observatorului.



Se efectuează proiecția perspectiva în planul din față (în fereastra vizuală), cu centrul de proiecție în poziția observatorului, a obiectelor cuprinse în volumul vizual. Obiectele situate în afara volumului vizual sunt eliminate sau decupate la frontiera volumului vizual.

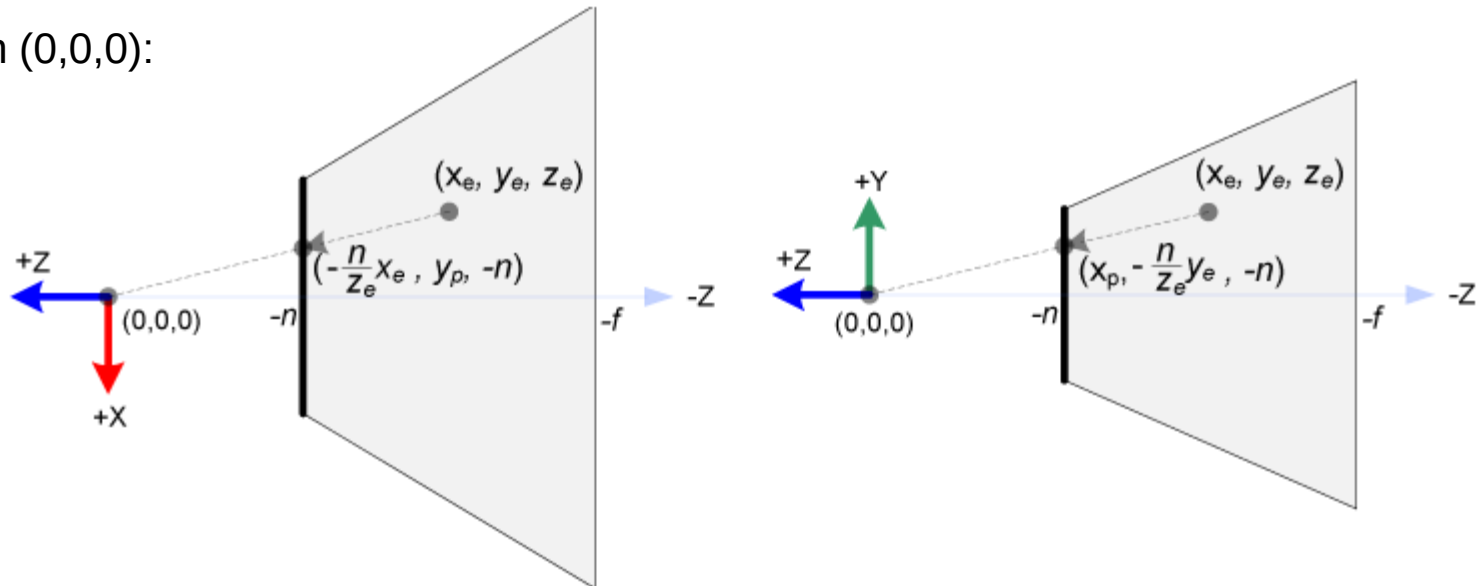
Transformarea de proiecție (4) - proiecția

- ❖ Proiecția se efectuează în fereastra vizuală din planul din față: planul $z = -\text{near}$ (în SCO).
- ❖ Varfurile proiectate sunt raportate la sist. coordonatelor observator.

Proiecția ortografică a unui punct din sist. coordonatelor observator, (x_e, y_e, z_e) , în fereastra vizuală:

$$x_p = x_e, y_p = y_e, z_p = -\text{near}.$$

Proiecția perspectivă a unui punct din SCO, (x_e, y_e, z_e) , în planul $z = -\text{near}$, cu centrul de proiecție în $(0,0,0)$:



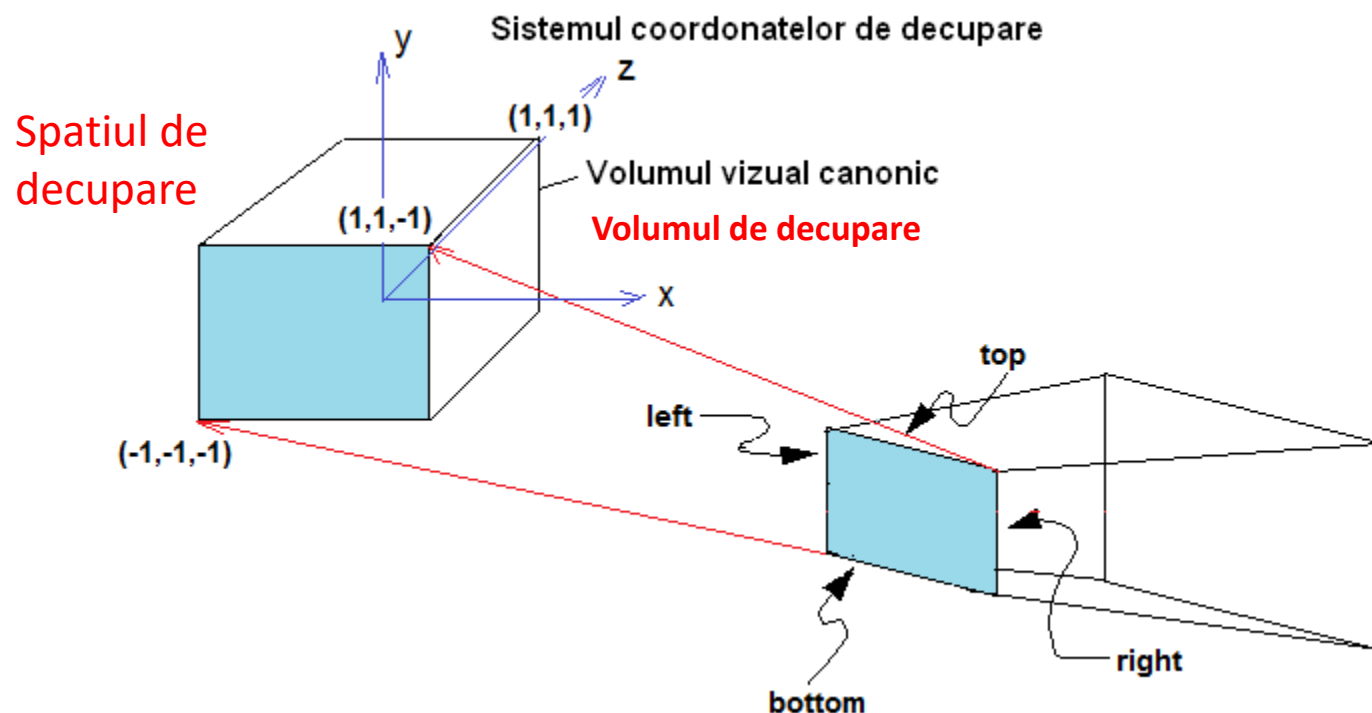
$x_p = (-\text{near}/z_e)*x_e$, $y_p = (-\text{near}/z_e)*y_e$ – se pot obtine aplicand formulele proiectiei perspectiva cu centrul de proiectie in $(0,0,0)$ si planul de proiectie este $z = -\text{near}$.

Transformarea de proiecție (5) – Sistemul coordonatelor de decupare

- ❖ Transformarea de proiecție este însoțită de operația de decupare a primitivelor grafice la marginile volumului vizual.
- ❖ Pentru simplificarea calculelor de decupare, volumul vizual definit de programator este transformat în volumul vizual canonic: cub cu latura de 2 unități, centrat în originea sistemului coordonatelor de decupare, cu laturile paralele cu axele.
- ❖ Transformarea prin care volumul vizual este transformat în volumul vizual canonic se numește transformarea de normalizare.
- ❖ În OpenGL transformarea de proiecție include: proiecția definită prin volumul de vizualizare compusă cu transformarea de normalizare.
= transformarea coordonatelor observator în coordonate de decupare.
- ❖ După transformarea varfurilor prin transformarea de proiecție, varfurile sunt unite pe baza topologiei definite de programator, formând primitive grafice 3D. Acestea sunt decupate la marginile volumului vizual canonic.

Transformarea de proiecție (6) - normalizarea

- *Sistemul coordonatelor de decupare*: sistem de coordonate carteziane 3D stanga.
- **Prin transformarea de normalizare**:
 - colțurile ferestrei 2D din planul de proiecție, **(left, bottom,-near)** și **(right, top,-near)** sunt mapate pe colțurile stânga-jos și dreapta-sus ale ferestrei 2D din sistemul coordonatelor de decupare, adică **$(-1,-1,-1)$** și **$(1,1,-1)$** ;
 - planul **$z = -near$** se mapeaza pe **$z = -1$** , iar **$z = -far$** se mapeaza pe **$z = 1$**



Transformarea de proiecție (7) calculul coordonatelor de decupare

- Impunand conditia $(\text{left}, \text{bottom}) \rightarrow (-1, -1)$ si $(\text{right}, \text{top}) \rightarrow (1, 1)$, rezulta transformarea punctelor proiectate pe planul din față, (x_p, y_p) , in sistemul coordonatelor de decupare – este o transformare freamstra-poarta: $x_p = (-\text{near}/z_e) * x_e$

$$x_d = 2 * x_p / (\text{right} - \text{left}) - (\text{right} + \text{left}) / (\text{right} - \text{left})$$

$$y_d = 2 * y_p / (\text{top} - \text{bottom}) - (\text{top} + \text{bottom}) / (\text{top} - \text{bottom})$$

Pentru proiectia perspectiva:

Inlocuind x_p si y_p prin expresiile lor in functie de (x_e, y_e, near) rezulta:

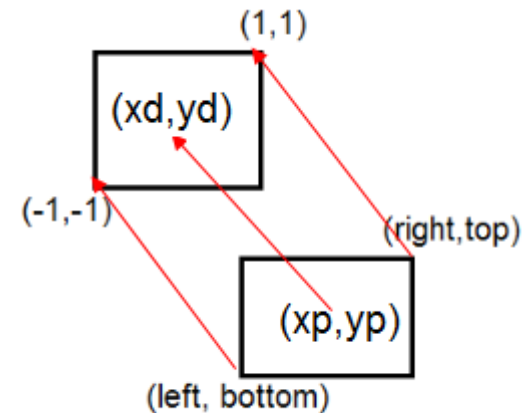
$$x_d = (x_e * 2 * \text{near} / (\text{right} - \text{left}) + z_e * (\text{right} + \text{left}) / (\text{right} - \text{left})) / (-z_e)$$

$$y_d = (y_e * 2 * \text{near} / (\text{top} - \text{bottom}) + z_e * (\text{top} + \text{bottom}) / (\text{top} - \text{bottom})) / (-z_e)$$

Pentru transformarea coordonatelor z se impun conditiile: $-\text{near} \rightarrow -1$, $-\text{far} \rightarrow 1$; rezulta:

$$z_d = (-z_e * (\text{far} + \text{near}) / (\text{far} - \text{near}) - 2 * \text{far} * \text{near} / (\text{far} - \text{near})) / (-z_e)$$

(x_d, y_d, z_d) sunt coordonate carteziane in sistemul coordonatelor de decupare (clip coordinates);



Matricile transformării de proiecție -1

Pentru proiecția perspectivă:

$$\text{Per} = \begin{bmatrix} \frac{2 * \text{near}}{\text{right} - \text{left}} & 0 & A & 0 \\ 0 & \frac{2 * \text{near}}{\text{top} - \text{bottom}} & B & 0 \\ 0 & 0 & C & D \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad \begin{aligned} A &= \frac{\text{right} + \text{left}}{\text{right} - \text{left}} & B &= \frac{\text{top} + \text{bottom}}{\text{top} - \text{bottom}} \\ C &= -\frac{\text{far} + \text{near}}{\text{far} - \text{near}} & D &= -\frac{2 * \text{far} * \text{near}}{\text{far} - \text{near}} \end{aligned}$$

$$[x_c \ y_c \ z_c \ w_c]^T = \text{Per} \cdot [x_e \ y_e \ z_e \ 1]^T \quad x_c = x_d * w_c, \ y_c = y_d * w_c, \ z_c = z_d * w_c, \ w_c = -z_e$$

Pentru proiecția ortografică:

$$x_p = x_e \text{ si } y_p = y_e \rightarrow x_d = 2 * x_e / (\text{right} - \text{left}) - (\text{right} + \text{left}) / (\text{right} - \text{left})$$

$$y_d = 2 * y_e / (\text{top} - \text{bottom}) - (\text{top} + \text{bottom}) / (\text{top} - \text{bottom})$$

$$\text{Par} = \begin{bmatrix} \frac{2}{\text{right} - \text{left}} & 0 & 0 & -A \\ 0 & \frac{2}{\text{top} - \text{bottom}} & 0 & -B \\ 0 & 0 & \frac{-2}{\text{far} - \text{near}} & C \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad [x_c \ y_c \ z_c \ 1]^T = \text{Par} \cdot [x_e \ y_e \ z_e \ 1]^T$$

Matricile transformării de proiecție -2

Matricile de transformări de proiecție pot fi obținute apelând următoarele funcții din biblioteca glm:

`glm::ortho(float left, float right, float bottom, float top, float zNear, float zFar);`

`glm::frustum (float left, float right, float bottom, float top, float zNear, float zFar);`

`glm::perspective (float fov, float aspect, float znear, float zfar);`

Creaza o matrice de proiecție pentru un volum perspectiva simetric:

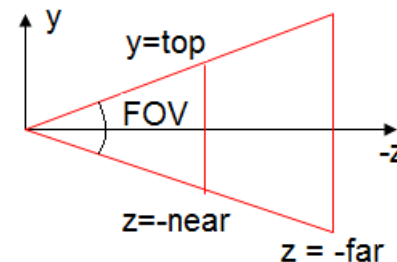
- fov: Field of View

$$\tan(\text{FOV}/2) = \text{top}/\text{near}$$

- aspect: aspect ratio

$$\text{aspect ratio} = (\text{right-left})/(\text{top-bottom})$$

= raportul dintre latimea si inaltimea ferestrei vizuale



Împartirea perspectivă

Transformarea de proiecție: coordonate observator → coordonate de decupare

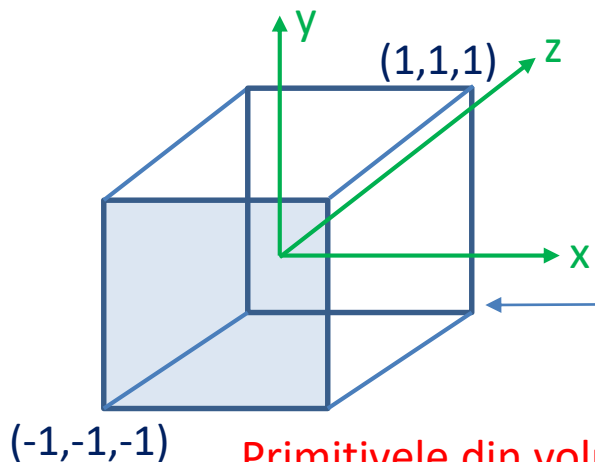
$$[x_c \ y_c \ z_c \ w_c]^T = \text{Par/Per} \cdot [x_e \ y_e \ z_e \ 1]^T \text{ (clip coordinates)} \leftarrow \text{eye coordinates}$$

Împartirea perspectivă: coordonate de decupare → coordonate dispozitiv normalizate

$$x_d = x_c/w_c, \ y_d = y_c/w_c, \ z_d = z_c/w_c$$

- Varfurile rezultate din împartirea perspectivă sunt asamblate în primitive grafice care sunt decupate la frontiera volumului vizual canonic.

Sistemul coordonatelor de decupare



După transformarea de proiecție, observatorul este la infinit pe axa OZ negativă a sistemului coord. de decupare.



Distanța față de observator se măsura pe axa OZ pozitivă.

Volumul de decupare

După decupare: $-1 \leq x_d, y_d, z_d \leq 1$

Primitivele din volumul de decupare sunt proiectate ortografic pe fața “din față” a volumului: $(-1, -1, 1) - (1, 1, 1)$.

Transformarea în poarta de afișare

coordonate dispozitiv normalizate $\rightarrow$ coordonate ecran

- Este o transformare fereasta-poarta, în care:
- Fereastra este fereastra 2D din sistemul coordonatelor de decupare având colțurile în $(-1, -1, -1)$ - $(1, 1, -1)$.

- Poarta este definită de programator folosind funcția:

`void glViewport(GLint px, GLint py, GLsizei width, GLsizei height);`

(px, py) reprezintă coordonate relative la colțul stanga-jos al fereastei aplicației (în pixeli). Valorile implicite : $(0,0)$.

width, height reprezintă lățimea/ înălțimea porții de afișare.

Valorile implicite sunt: lățimea și înălțimea ferestrei aplicației.

- Transformarea în poarta de afișare este definită astfel:

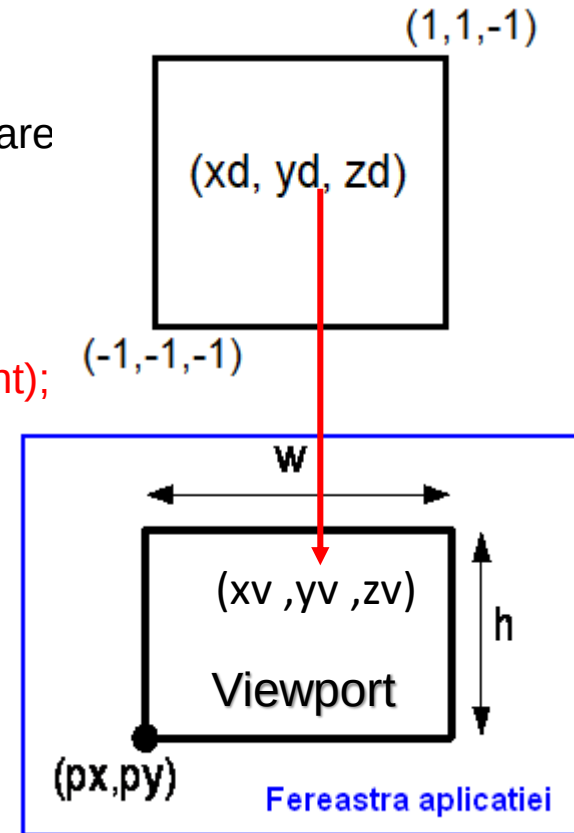
$$xv = (width/2)xd + ox, \quad ox = px + width/2$$

$$yv = (height/2)yd + oy, \quad oy = py + height/2$$

$$zv = ((f-n)/2)zd + (n+f)/2 \quad // \text{ conserva coordonata } z \text{ pentru eliminarea partilor nevizibile}$$

n (near) și f (far) au valorile implicite 0.0 respectiv 1.0, pentru care $0 \leq zv \leq 1$

n și f pot fi modificate la valori cuprinse în intervalul $[0,1]$ folosind funcția **DepthRange()**



Transformări Geometrice 2D

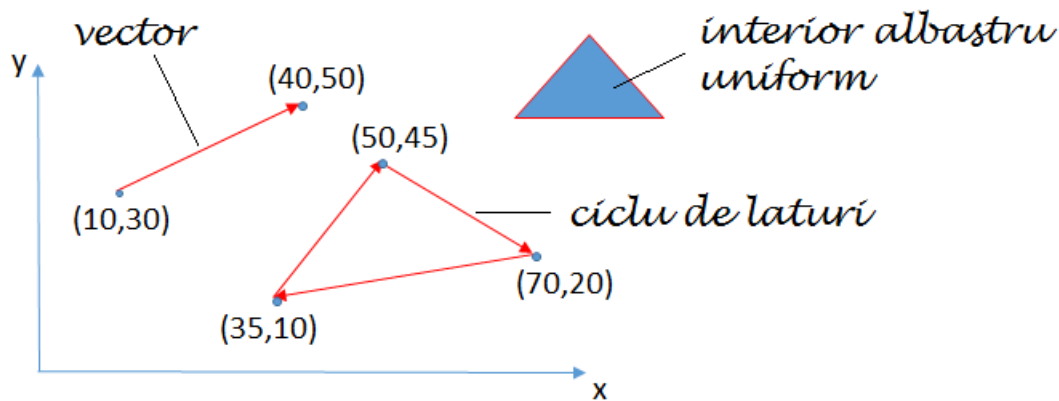
Prof. univ. dr. ing. Florica Moldoveanu

Curs Elemente de Grafică pe Calculator – UPB, Automatică și Calculatoare
2019-2020

Transformari geometrice(1)

Obiectele 2D/3D sunt reprezentate prin:

- **Date geometrice:** coordonatele varfurilor, raportate la un sistem de coordonate carteziane 2D sau 3D;
- **Atribute topologice,** care definesc modul in care sunt conectate varfurile (laturi, ciclul de laturi al unei fețe, ș.a.);
- **Atribute de aspect:** culoarea, tipul de interior pentru suprafete 2D, atribute de material pentru suprafete 3D (ex: reflexia/refractia luminii de catre suprafata), s.a.



Transformările
geometrice se aplica
coordonatelor varfurilor
obiectului si nu afecteaza
atributele sale!

Transformari geometrice (2)

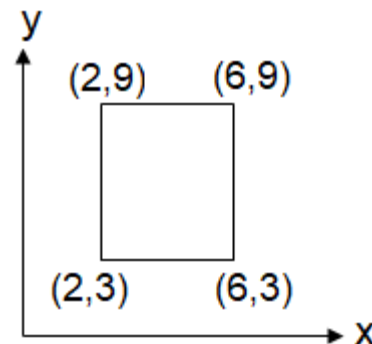
- Sunt operatii fundamentale in sinteza imaginilor
- Folosite pentru:
 - Redarea desenelor/obiectelor 2D/3D la diferite marimi
 - Compunerea desenelor/scenelor 3D din mai multe obiecte
 - Realizarea animatiei
 - Transformarea obiectelor dintr-un sistem de coordonate in alt sistem de coordonate
 - Etc.

Transformări geometrice 2D (1)

Sunt transformări ale obiectelor definite în sistemul de coordonate cartezienne XOY, numite și obiecte 2D. Fiecare vârf al unui obiect 2D este definit printr-o pereche de coordonate (x,y) .

Transformările geometrice 2D elementare

- Translația
- Scalarea față de origine
- Rotația față de origine
- Forfecarea față de origine
- Oglindirile față de axe principale și față de origine



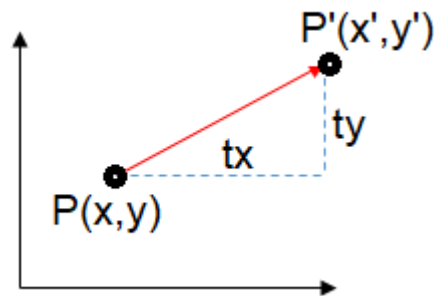
Orice altă transformare se obține prin compunerea a 2 sau mai multe transformări elementare

- Scalarea /rotația compusă cu translație
- Forfecare față de un punct oarecare din plan, etc

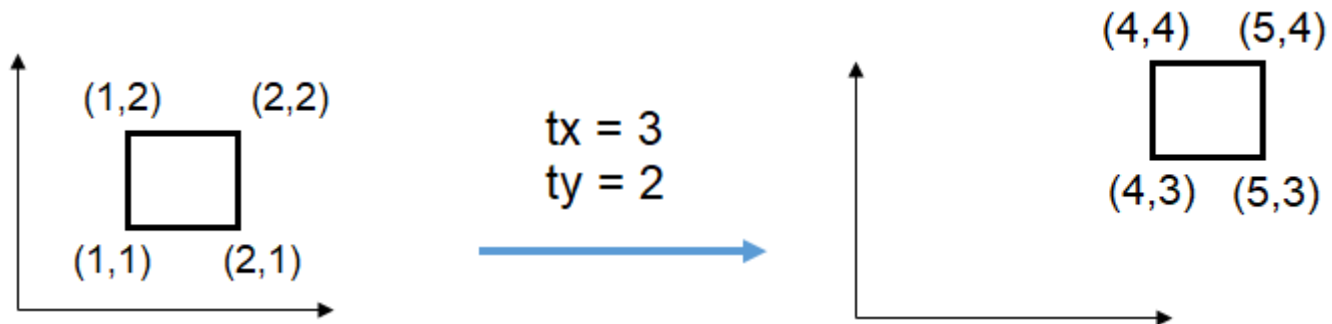
Transformări geometrice 2D elementare (1)

- Translatia (1)

Este definita printr-un vector, $T[tx,ty]$.

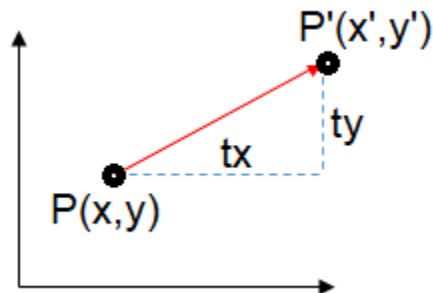


$$\begin{aligned}x' &= x + tx \\ y' &= y + ty\end{aligned}$$



Transformari geometrice 2D elementare (2)

• Translatia (2)



$$\begin{aligned}x' &= x + tx \\ y' &= y + ty\end{aligned}$$

- Se dorește o reprezentare matricială a transformării, pentru compunerea sa cu alte transformări, de ex. translație și scalare: compunerea transformărilor se realizează prin înmulțirea matricilor transformărilor elementare.
- Un punct din plan, $P(x, y)$, se reprezintă în coordonate carteziene printr-un vector $[x, y]$ sau $\begin{bmatrix} x \\ y \end{bmatrix}$

→ Nu există o matrice M , de 2×2 , astfel încât să exprimăm translația prin:

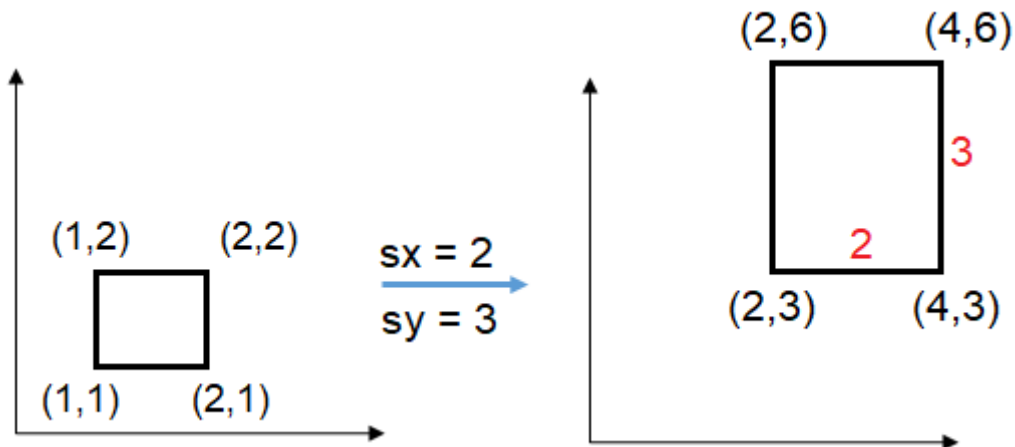
$$[x', y'] = [x, y] * M \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = M * \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformari geometrice 2D elementare (3)

Scalarea fata de origine

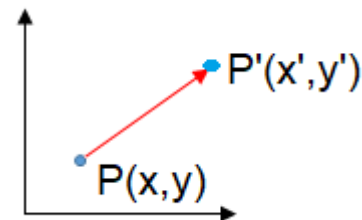
Este definita prin 2 numere reale, de regula pozitive:

- s_x - scalarea pe axa OX
- s_y - scalarea pe axa OY



Efecte:

- Daca s_x sau s_y sau ambii sunt >1 :
- Daca s_x sau s_y sau ambii sunt <1 :.....
- $s_x = s_y$, scalare uniforma



$$\begin{aligned}x' &= x * s_x \\ y' &= y * s_y\end{aligned}$$

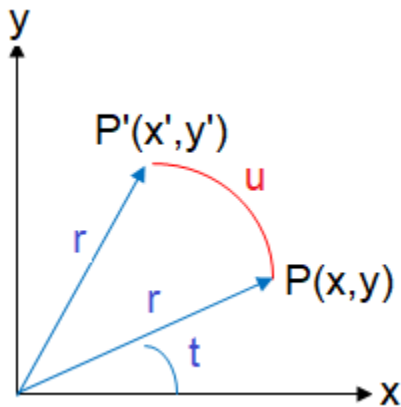
Reprezentarea matriciala

$$\begin{bmatrix} x' & y' \end{bmatrix} = \begin{bmatrix} x & y \end{bmatrix} * \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformări geometrice 2D elementare (4)

Rotația fata de origine



$x = r \cdot \cos(t)$
 $y = r \cdot \sin(t)$

Relatia dintre coordonatele carteziane si coordonatele polare ale unui punct $P(x, y)$

$$x' = r \cdot \cos(t+u) = r \cdot (\cos(t) \cdot \cos(u) - \sin(t) \cdot \sin(u)) = x \cdot \cos(u) - y \cdot \sin(u)$$
$$y' = r \cdot \sin(t+u) = r \cdot (\cos(t) \cdot \sin(u) + \sin(t) \cdot \cos(u)) = x \cdot \sin(u) + y \cdot \cos(u)$$

Rotatia fata de origine

$$x' = x \cdot \cos(u) - y \cdot \sin(u)$$

$$y' = x \cdot \sin(u) + y \cdot \cos(u)$$

$$\begin{bmatrix} x' & y' \end{bmatrix} = \begin{bmatrix} x & y \end{bmatrix} \cdot \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) \\ \sin(u) & \cos(u) \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformări geometrice 2D fata de un punct oarecare din plan (1)

Scalarea fata de un punct oarecare din plan

- Punctul fix al transformării este un punct oarecare $F(x_f, y_f)$ – coordonatele sale nu se modifica prin aplicarea transformării.
- $P(x, y)$ este punctul transformat
- **Scalarea se aplica vectorului FP $[x-x_f, y-y_f]$. Prin scalare se transforma în FP' $[x'-x_f, y'-y_f]$:**

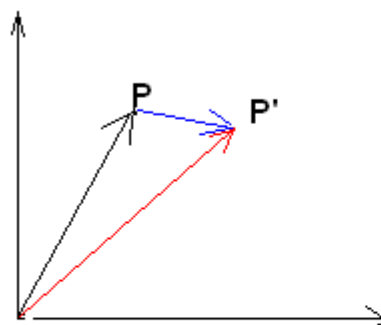
$$x' - x_f = s_x(x - x_f)$$

$$y' - y_f = s_y(y - y_f)$$

Rezultă:

$$x' = x \cdot s_x + x_f - x_f \cdot s_x$$

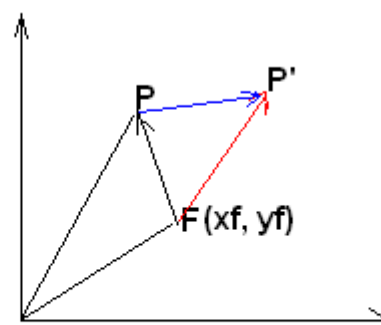
$$y' = y \cdot s_y + y_f - y_f \cdot s_y$$



Scalarea fata de origine

$$x' - 0 = s_x(x - 0)$$

$$y' - 0 = s_y(y - 0)$$



Scalarea fata de un punct F din plan

$$x' - x_f = s_x(x - x_f)$$

$$y' - y_f = s_y(y - y_f)$$

Nu poate fi exprimata în coordonate carteziane printr-o matrice de 2x2!

Transformări geometrice 2D fata de un punct oarecare din plan(2)

Rotatia fata de un punct oarecare din plan

- Punctul fix al transformarii este $F(x_f, y_f)$
- $P(x, y)$ este punctul transformat
- **Rotatia se aplica vectorului FP $[x - x_f, y - y_f]$, in jurul lui F .**

Rezulta vectorul FP' $[x' - x_f, y' - y_f]$

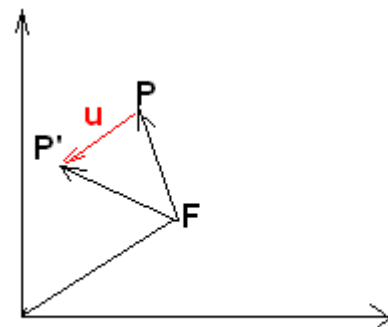
$$x' - x_f = (x - x_f) * \cos(u) - (y - y_f) * \sin(u)$$

$$y' - y_f = (x - x_f) * \sin(u) + (y - y_f) * \cos(u)$$

Rezultă:

$$x' = x * \cos(u) - y * \sin(u) + x_f - x_f * \cos(u) + y_f * \sin(u)$$

$$y' = x * \sin(u) + y * \cos(u) + y_f - x_f * \sin(u) - y_f * \cos(u)$$



Rotatia fata de un punct F din plan

Nu poate fi exprimata in coordonate carteziane printr-o matrice de 2x2!

Compunerea transformărilor geometrice

De ce este necesară?

- Pentru a putea aplica o singura transformare care înglobează o secvență de transformări elementare, în locul aplicării fiecărei transformări din secvență; de ex., se aplică tuturor vârfurilor o transformare care înglobează scalare (S), rotație (R) și translație (T) în loc să se aplice fiecărui vârf secvența de transformări elementare.

$P_t = M * P$ sau $P_t = P * M^T$, unde M^T este transpusa matricei M

în loc de $P' = S * P$; $P'' = R * P'$; $P_t = T * P''$ unde $M = T * R * S$

- Matricea unei transformări compuse se obține prin înmulțirea matricilor transformărilor elementare. Exemplu:

$$SR = S * R = \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix} \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

$$RS = S * R = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

Atentie: $RS \neq SR$

- **Translația nu poate fi reprezentată în coordonate carteziane printr-o matrice!**
- **Aceasta impune reprezentarea transformărilor în coordonate omogene.**

Reprezentarea transformărilor geometrice 2D în coordonate omogene(1)

Coordonate omogene:

Un punct din plan, $P(x,y)$, se reprezintă în coordonate omogene printr-un vector

$$[xw, yw, w] \quad \text{sau} \quad \begin{bmatrix} xw \\ yw \\ w \end{bmatrix} \quad xw = x * w; yw = y * w; w - \text{orice număr real}$$

Exemplu: $P(2, 0.5) \rightarrow [2, 0.5, 1], [4, 1, 2], [20, 5, 10]$

Transformarea din coordonate omogene în coordonate carteziene:

$[xw \ yw \ w] \rightarrow P(x, y)$, unde:

- pentru $w \neq 0$, $x = xw/w$, $y = yw/w$
- pentru $w = 0$, P este un punct la infinit

Exemple: $[1, 0, 0]$ este un punct la infinit pe direcția axei OX,

$[a, a, 0]$ este un punct la infinit pe direcția dreptei $[0,0] \rightarrow [a,a]$

Reprezentarea transformărilor elementare 2D în coordonate omogene

Translația

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

In OpenGL se folosesc vectori coloană pentru a reprezenta puncte din plan sau din spațiu.

Scalarea față de origine

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotăția față de origine

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} \cos(u) & \sin(u) & 0 \\ -\sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) & 0 \\ \sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Transformările inverse ale transformărilor elementare

$T(t_x, t_y)$: matricea translației, în coordonate omogene

$S(0, 0, s_x, s_y)$: matricea scalariei față de origine, în coordonate omogene

$R(0,0,u)$: matricea rotației față de origine, în coordonate omogene

Transformările inverse sunt:

$$T(t_x, t_y)^{-1} = T(-t_x, -t_y)$$

$$S(0, 0, s_x, s_y)^{-1} = S(0, 0, 1/s_x, 1/s_y)$$

$$R(0,0, u)^{-1} = R(0, 0, -u)$$

Transformari geometrice 2D compuse(1)

Exemple de transformari compuse: scalarea/rotatia fata de un punct oarecare din plan.

➤ Matricea transformarii compuse se obtine prin înmulțirea matricilor urmatoarelor transformari elementare:

- Translația prin care punctul fix al transformării ajunge în origine: $T(-x_f, -y_f)$;
- Scalarea / rotația față de origine: $S(0,0,s_x,s_y)/R(0,0,u)$;
- Translația inversă celei de la punctul 1: $T(x_f, y_f)$.

$$[x' \ y' \ 1] = [x \ y \ 1] * T(-x_f, -y_f) * S(0,0,s_x,s_y)/R(0,0,u) * T(x_f, y_f)$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = T(x_f, y_f) * S(0,0,s_x,s_y)/R(0,0,u) * T(-x_f, -y_f) * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Transformări geometrice 2D compuse(2)

Scalarea față de punctul F(xf,yf)

$$[x' \ y' \ 1] = [x \ y \ 1] \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_f & -y_f & 1 \end{bmatrix}}_{T(-x_f, -y_f)} \underbrace{\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{S(0, 0, s_x, s_y)} \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_f & y_f & 1 \end{bmatrix}}_{T(x_f, y_f)}$$

Matricea transformării compuse:

$$M = T(-x_f, -y_f) * S(0, 0, s_x, s_y) * T(x_f, y_f)$$

Aplicarea transformării compuse:

$$[x' \ y' \ 1] = [x \ y \ 1] * M$$

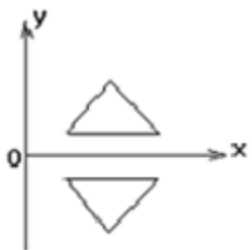
Alte transformari geometrice 2D(1)

Simetria (Oglindirea)

Față de axa OX:

$$x' = x$$

$$y' = -y$$

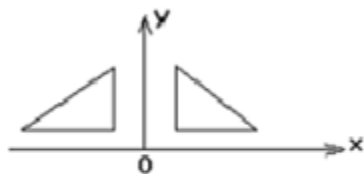


$$[x' y' 1] = [x y 1] \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Față de axa OY:

$$x' = -x$$

$$y' = y$$

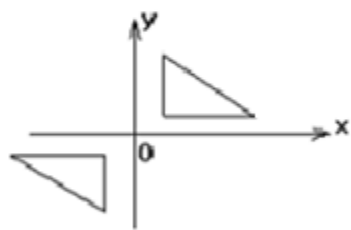


$$[x' y' 1] = [x y 1] \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Față de origine:

$$x' = -x$$

$$y' = -y$$

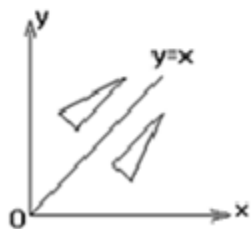


$$[x' y' 1] = [x y 1] \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Față de dreapta $y=x$:

$$x' = y$$

$$y' = x$$



$$[x' y' 1] = [x y 1] \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Alte transformari geometrice elementare 2D(2)

Oglindirea față de o dreaptă oarecare

Se exprimă ca transformare compusa prin înmulțirea matricilor urmatoarelor transformari:

1. O translație, astfel încât dreapta sa treaca prin origine: $T(tx,ty)$
2. O rotație față de origine, a.î. dreapta să se suprapună peste una dintre axele principale:
 $R(0,0,u)$
3. Oglindirea față de axa principală peste care a fost suprapusă dreapta: O
4. Rotația inversă celei de la punctul 2: $R^{-1} = R(0,0,-u)$
5. Translația inversă celei de la punctul 1: $T^{-1} = T(-tx,ty)$

În notație matricială: $\mathbf{M} = \mathbf{T} * \mathbf{R} * \mathbf{O} * \mathbf{R}^{-1} * \mathbf{T}^{-1}$ (folosind vectori linie) sau

$$\mathbf{M} = \mathbf{T}^{-1} * \mathbf{R}^{-1} * \mathbf{O} * \mathbf{R} * \mathbf{T} \text{ (folosind vectori coloana)}$$

Deduceti elementele matricilor T , R , O , atunci cand dreapta este data printr-un punct, (x_d, y_d) și o directie, $D[a, b]$.

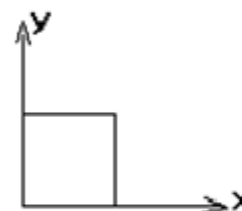
Alte transformari geometrice 2D (3)

Forfecarea

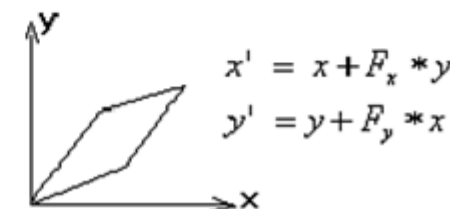
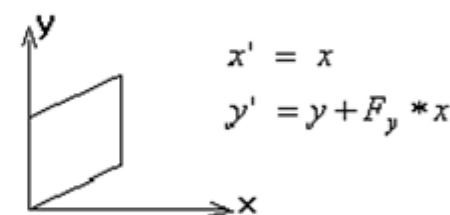
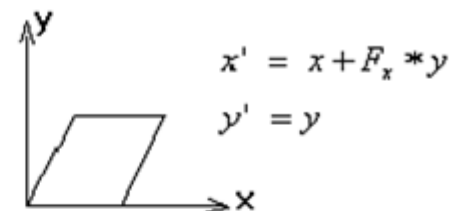
Forfecarea față de origine – transformare elementara- este definita prin 2 numere reale:

F_x : factorul de forfecare pe axa OX

F_y : factorul de forfecare pe axa OY



Forfecarea
fata de origine



Deduceti formele matriciale ale transformarilor de forfecare față de origine

Forfecarea fata de un punct oarecare din plan, (x_f, y_f) , exprimata ca transformare compusa:

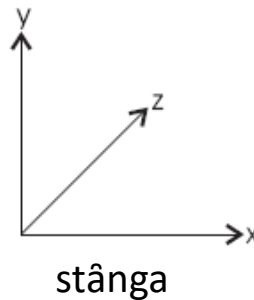
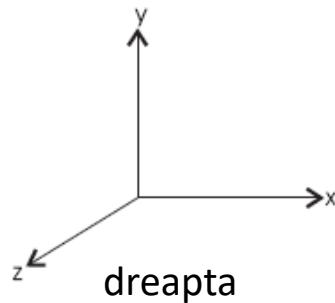
1. Translatie prin care punctul (x_f, y_f) ajunge in origine
2. Forfecarea fata de origine
3. Translatia inversa celei de la pasul 1

Transformări geometrice 3D

Prof. univ. dr. ing. Florica Moldoveanu

Curs Elemente de Grafică pe Calculator – UPB, Automatică și Calculatoare
2019-2020

Sisteme de coordonate carteziane 3D



Sisteme de coordonate carteziane 3D

i, j, k : versorii direcțiilor axelor sistemului de coordonate

dreapta: $k = (i \times j) / |i \times j|$

stanga: $k = (j \times i) / |j \times i|$

Sistemul de coordonate în care este descrisa scena într-o aplicatie OpenGL: sistem de coordonate carteziane 3D dreapta.

Transformările geometrice 3D elementare

- Translația
- Scalarea față de origine
- Rotațiile în jurul axelor principale ale sistemului de coordonate
- Oglindirea față de un plan principal al sistemului de coordonate
- Forfecarea față de originea sistemului de coordonate

Considerăm punctele din spațiu reprezentate prin vectori coloană, pentru a respecta convenția din OpenGL:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = M \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

M este matricea de transformare a punctului (x, y, z), în coordonate omogene

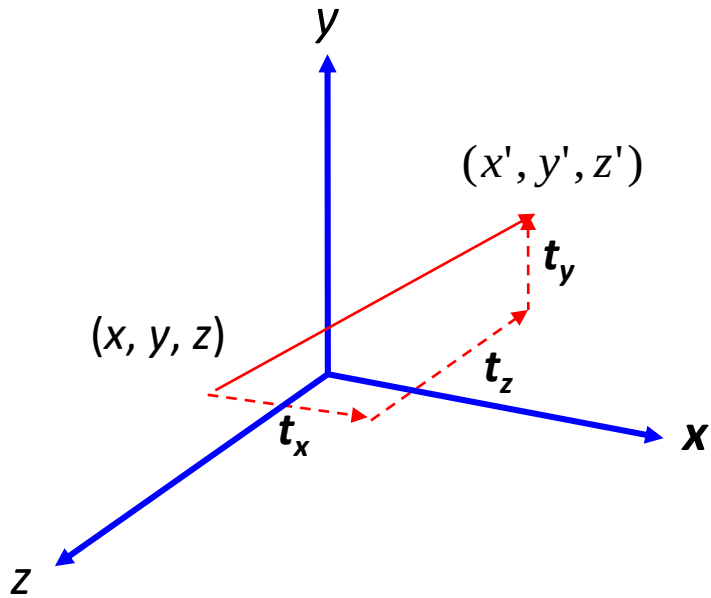
Translația

Definita printr-un vector $T[t_x, t_y, t_z]$

$$x' = x + t_x$$

$$y' = y + t_y$$

$$z' = z + t_z$$



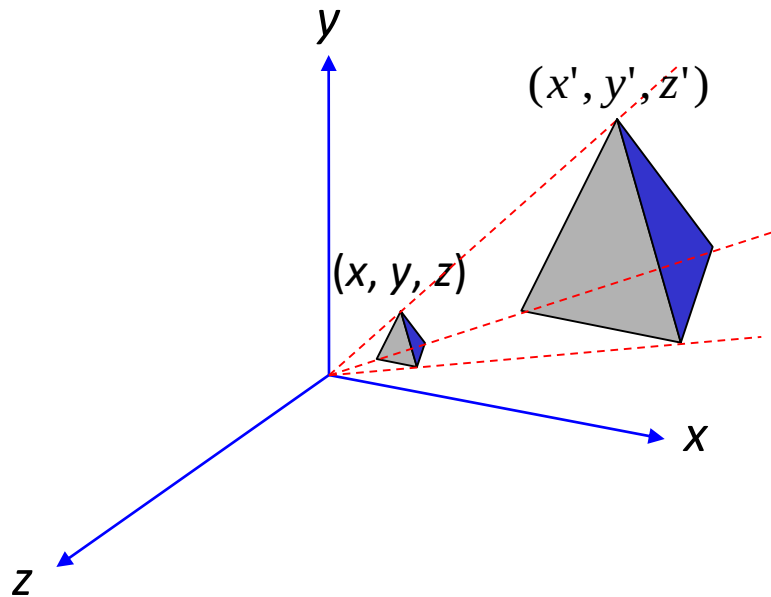
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$$\uparrow \\ T(t_x, t_y, t_z)$$

Scalarea față de origine

Definita prin 3 numere reale: S_x, S_y, S_z
– factorii de scalare pe cele 3 axe

$S_x = S_y = S_z \Rightarrow$ scalare uniforma
altfel, scalare neuniforma



$$x' = x \cdot S_x$$

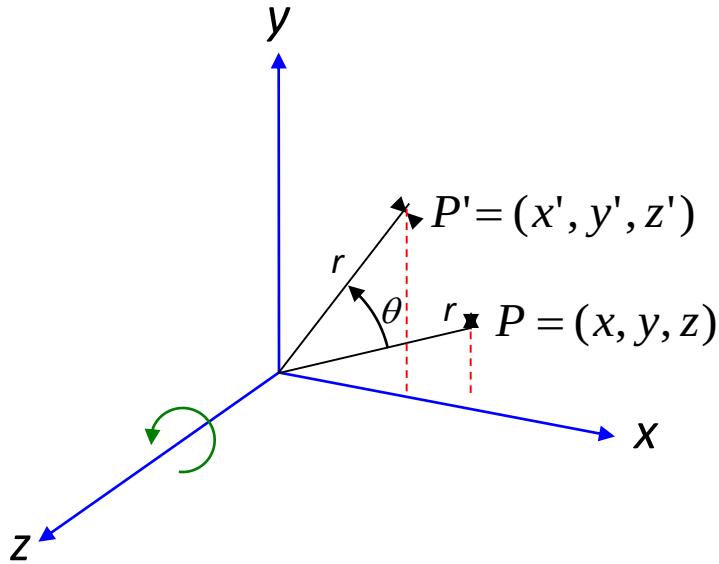
$$y' = y \cdot S_y$$

$$z' = z \cdot S_z$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$$\uparrow \\ S(s_x, s_y, s_z)$$

Rotația pozitivă (trigonometrică) în jurul axei oz



Este o rotație într-un plan de z constant (prin rotație nu se modifică coordonata z).

Rotatia in planul XOY:

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

$$z' = z = 0$$



$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$$R_z(\theta)$$

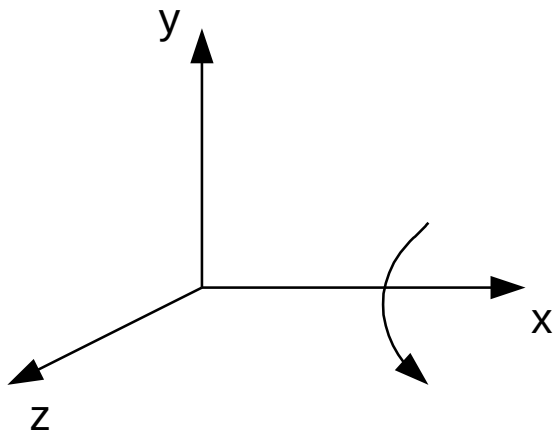
Rotatia punctului $P(x,y,z)$ in jurul axei Oz cu unghiul θ :

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

$$z' = z$$

Rotația pozitivă în jurul axei ox

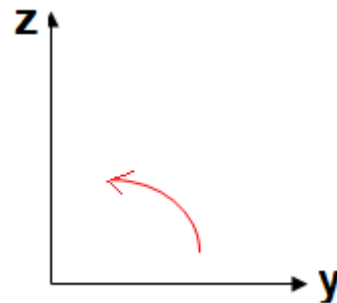


Rotatia in planul YOZ

$$y' = y \cos \theta - z \sin \theta$$

$$z' = y \sin \theta + z \cos \theta$$

$$x' = x = 0$$



Rotatia punctului $P(x,y,z)$ in jurul axei Ox cu unghiul θ :

$$x' = x$$

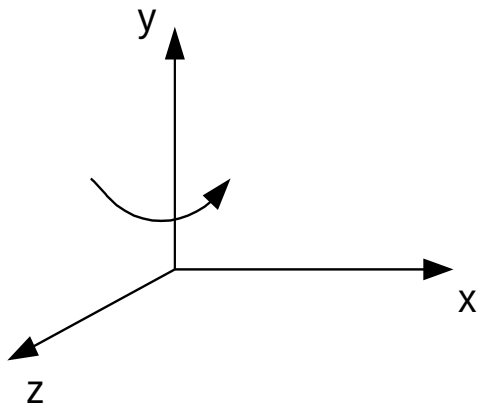
$$y' = y \cos \theta - z \sin \theta$$

$$z' = y \sin \theta + z \cos \theta$$

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R_x(\theta)$

Rotația pozitivă în jurul axei oy

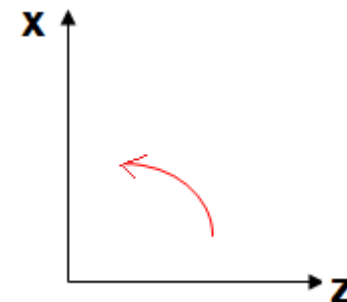


Rotatia in planul ZOx

$$z' = z \cos \theta - x \sin \theta$$

$$x' = x \cos \theta + z \sin \theta$$

$$y' = y = 0$$



Rotatia punctului $P(x,y,z)$ in jurul axei Oy cu unghiul θ :

$$x' = x \cos \theta + z \sin \theta$$

$$y' = y$$

$$z' = -x \sin \theta + z \cos \theta$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y(\theta)$$

Alte transformari geometrice 3D elementare

Oglindirea față de un plan principal al sistemului de coordonate

$$O_{xoy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Forfecarea față de origine

- **De-a lungul axei OZ: modifica x si y proportional cu z; F_x , F_y - factorii de forfecare**

$$x' = x + F_x * z$$

$$y' = y + F_y * z$$

$$z' = z$$



- **Analog pentru forfecarea de-a lungul axei OX si a axei OY**

- **Cazul general (forfecarea pe toate cele 3 axe):**

$$x' = x + y * d + z * g$$

$$y' = x * b + y + z * i$$

$$z' = x * c + y * f + z$$

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} * \begin{bmatrix} 1 & b & c & 0 \\ d & 1 & f & 0 \\ g & i & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Transformari geometrice 3D compuse

Exemple:

1. Scalarea față de un punct oarecare, F(xf, yf, zf):

$$T(x_f, y_f, z_f) * S(s_x, s_y, s_z) * T(-x_f, -y_f, -z_f)$$

2. Rotatia cu unghiul u in jurul unei axe paralele cu o axa a sistemului de coordonate:

1. Translatia obiectului astfel incat axa de rotatie sa se suprapuna peste o axa a sistemului de coordonate.
2. Rotatia obiectului in jurul axei sistemului de coordonate - R(u): $R_x(u)/R_y(u)/R_z(u)$
3. Translatia inversa celei din pasul 1.

Rezulta:

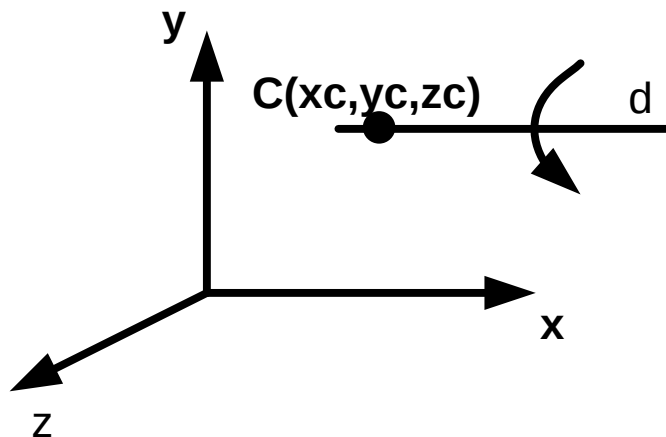
$$M = T(x_d, y_d, z_d) * R(u) * T(-x_d, -y_d, -z_d)$$

u : unghiul de rotatie

x_d, y_d, z_d : un punct de pe axa de rotatie

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = M * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Rotația în jurul unei drepte paralele cu axa OX



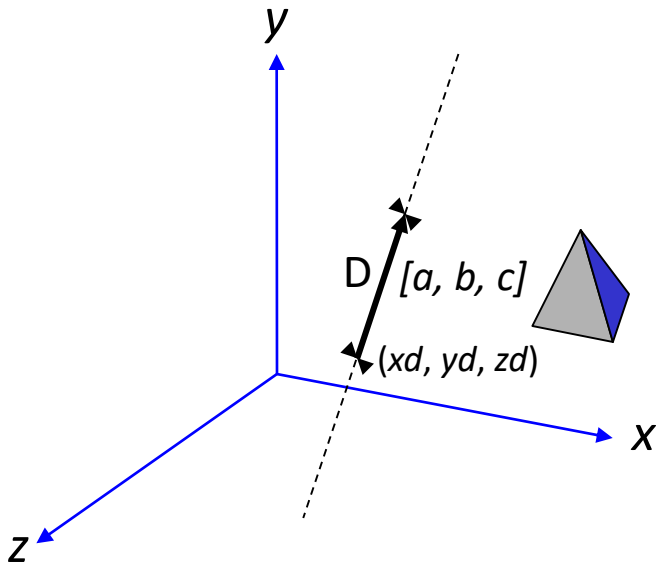
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = T(x_c, y_c, z_c) R_x(u) T(-x_c, -y_c, -z_c) \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Rotatia cu în jurul unei drepte oarecare (1)

Se considera dreapta data printr-un punct (x_d, y_d, z_d) si directia sa, $D[a, b, c]$.

1. Translatie prin care dreapta va trece prin origine: $T(-x_d, -y_d, -z_d)$
2. Alinierea dreptei cu una dintre axele principale, de ex. cu axa OZ:
 - 2.1. Rotatie in jurul axei OX, cu un unghi u_x , prin care dreapta ajunge in planul XOZ: $R_x(u_x)$
 - 2.2. Rotatie in jurul axei OY, cu un unghi u_y , prin care dreapta se suprapune pe axa OZ: $R_y(u_y)$
3. Rotatia cu unghiul dat, u , in jurul axei pe care s-a aliniat dreapta: rotatie in jurul axei OZ : $R_z(u)$
4. Transformarea inversa celei din pasul 2:
 - 4.1. Rotatie in jurul axei OY, cu unghiul $-u_y$: $R_y(-u_y)$
 - 4.2. Rotatie in jurul axei OX, cu unghiul $-u_x$: $R_x(-u_x)$
5. Transformarea inversa celei de la pasul 1: $T(x_d, y_d, z_d)$

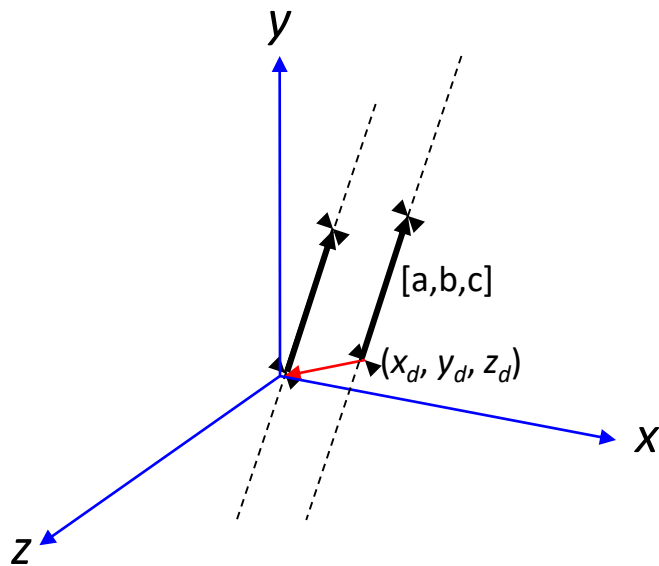
Rotatia in jurul unei drepte oarecare (2)



1. Translatie astfel incat dreapta sa treaca prin origine
2. Aliniere dreapta cu una din axe
3. Rotatia propriu-zisa.
4. Transformarea inversa de la punctul 2
5. Transformarea inversa de la punctul 1

Rotatia in jurul unei drepte oarecare (3)

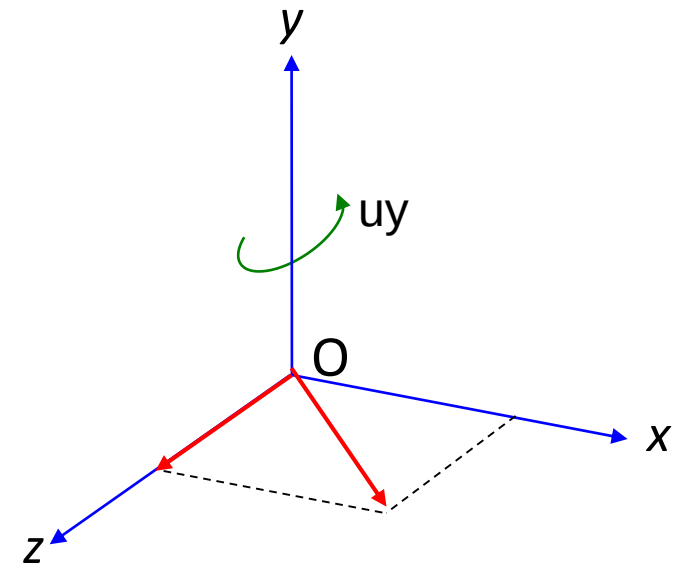
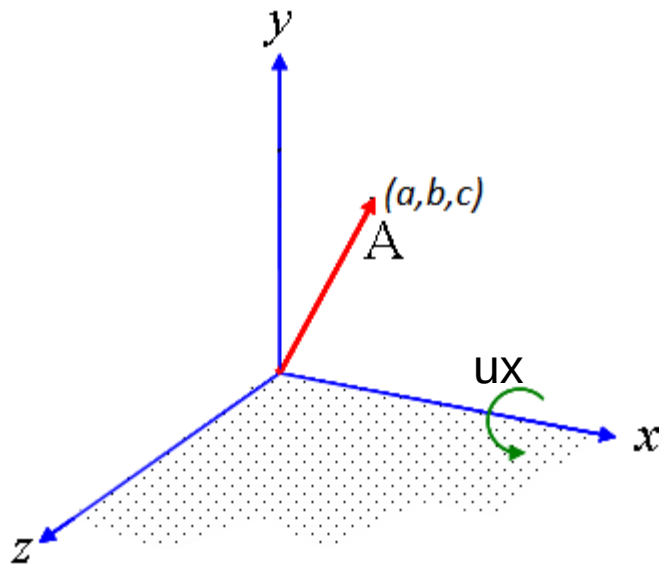
1. Translație



$$T = \begin{bmatrix} 1 & 0 & 0 & -x_d \\ 0 & 1 & 0 & -y_d \\ 0 & 0 & 1 & -z_d \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotatia in jurul unei drepte oarecare (4)

2. Aliniere D cu axa OZ

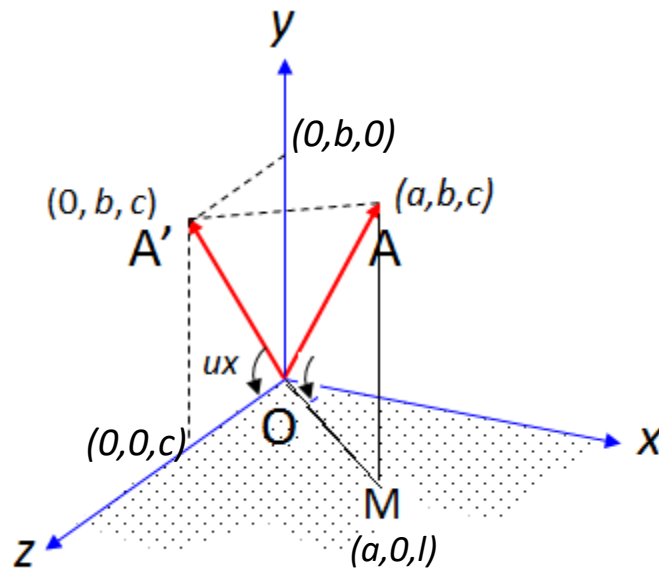


Rotatie in jurul axei OX cu un unghi ux , prin care dreapta ajunge in planul XOZ : $R_x(ux)$

Rotatie in jurul axei OY , cu un unghi uy , prin care dreapta se suprapune pe axa OZ : $R_y(uy)$

Rotatia in jurul unei drepte oarecare (5)

2. 1. Rotatia in jurul axei OX



$R_x(ux)$

Prin rotatia lui OA in jurul axei Ox cu unghiul ux , acesta va ajunge in planul XOZ

A' – proiectia lui A in planul YOZ

Lungime (OA) = L; lungime (OA') = l;

$$L = \sqrt{a^2 + b^2 + c^2}$$

$$l = \sqrt{b^2 + c^2}$$

$$b = l \cdot \sin(ux)$$

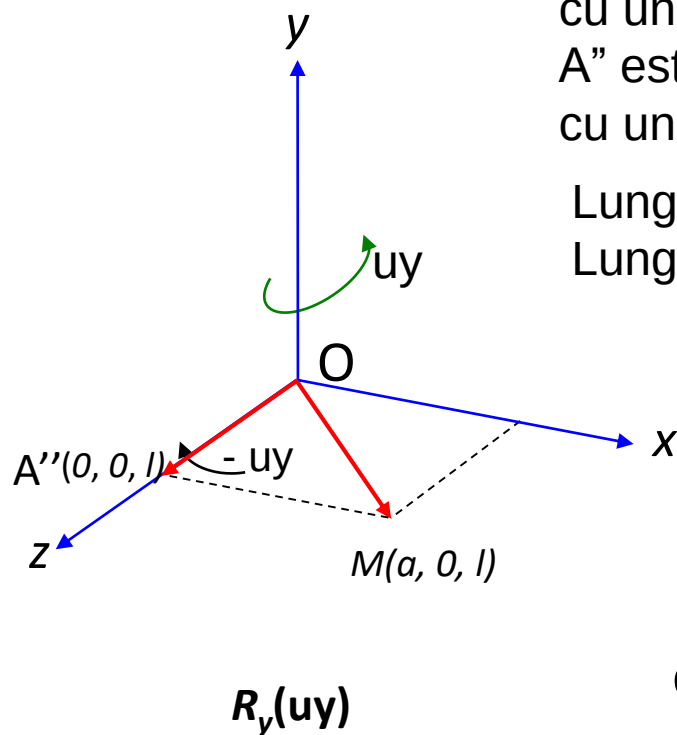
$$c = l \cdot \cos(ux)$$

$$\sin(ux) = \frac{b}{l}$$

$$\cos(ux) = \frac{c}{l}$$

Rotatia in jurul unei drepte oarecare (6)

2. 2. Rotatia in jurul axei OY



M este punctul obtinut prin rotatia lui A in jurul axei Ox, cu unghiul ux .

A'' este punctul obtinut prin rotatia lui A' in jurul axei Ox, cu unghiul ux

Lungime (OM) = Lungime (OA) = L

Lungime (OA'') = Lungime (OA') = l

$$a = L \cdot \sin(-uy)$$

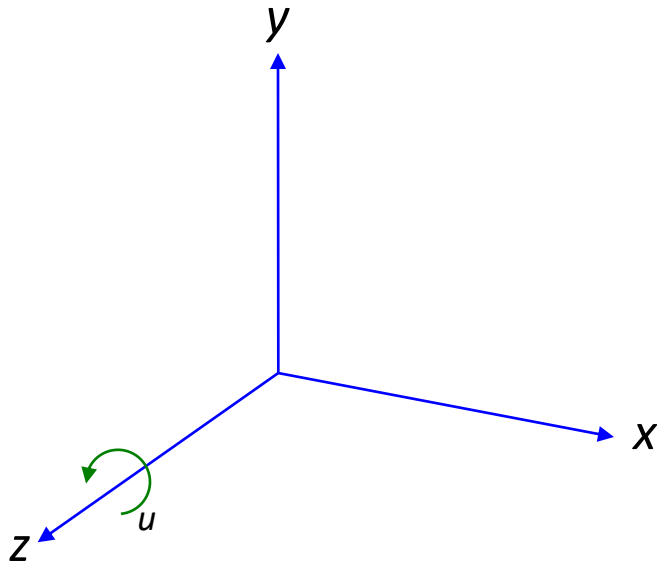
$$l = L \cdot \cos(-uy)$$

$$\sin(-uy) = a / L \rightarrow \sin(uy) = -a / L$$

$$\cos(-uy) = \cos(uy) = l / L$$

Rotatia in jurul unei drepte oarecare (7)

3. Rotatia in jurul axei OZ cu unghiul dat

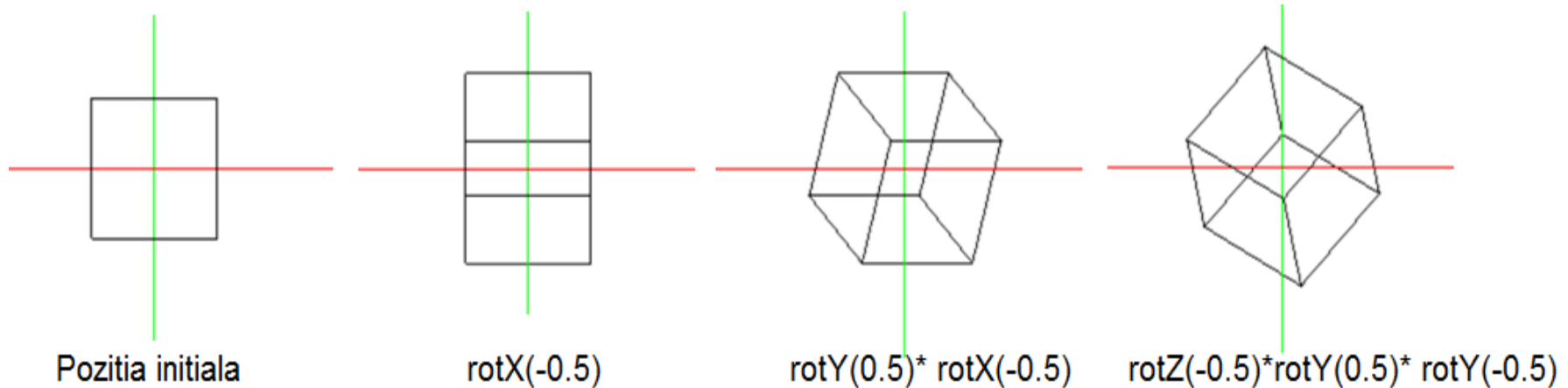


$$R_z(u) = \begin{bmatrix} \cos(u) & -\sin(u) & 0 & 0 \\ \sin(u) & \cos(u) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Exemplu: rotatia unui cub (1)

Cub centrat in originea sistemului de coordonate XYZ

Proiectie ortografica in XOY: $(x,y,z) \rightarrow (x' = x; y' = y)$



Exemplu: rotatia unui cub (2)

Cub cu centrul in (100, 100, 0)

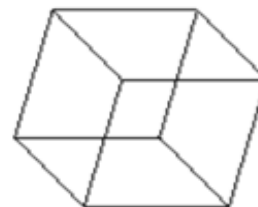
Proiectie ortografica in XOY



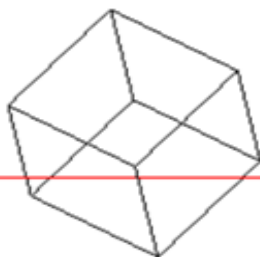
Pozitia initiala



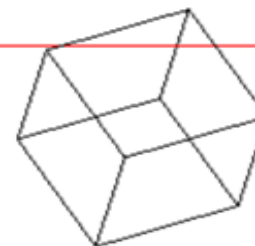
$\text{rotX}(-0.5)$



$\text{rotY}(0.5)*\text{rotX}(-0.5)$



$\text{rotZ}(-0.5)*\text{rotY}(0.5)*\text{rotX}(-0.5)$



$\text{rotZ}(-0.5)*\text{rotZ}(-0.5)*\text{rotY}(0.5)*\text{rotX}(-0.5)$

Implementarea transformărilor 3D – exemplu (1)

```
void AnimatieTransformari3D::Init()
{
    polygonMode = GL_FILL;
    Mesh* mesh = new Mesh("box");
    mesh->LoadMesh(RESOURCE_PATH::MODELS + "Primitives", "box.obj");
    meshes[mesh->GetMeshID()] = mesh;
    auto camera = GetSceneCamera();
    camera->SetPositionAndRotation(glm::vec3(0, 4, 10), glm::quat(glm::vec3(-20 *
        TO_RADIANS, 0, 0)));
    camera->Update();
    // initializare tx, ty and tz (pasii de translatie)
    translateX = 0; translateY = 0; translateZ = 0;
    // initialize sx, sy and sz (factorii de scalare)
    scaleX = 1; scaleY = 1; scaleZ = 1;
    // initializare pasi de rotatie
    angularStepOX = 0; angularStepOY = 0; angularStepOZ = 0;
}
```


Implementarea transformarilor 3D – exemplu (2)

```
void AnimatieTransformari3D::FrameStart()
{
    // clears the color buffer and depth buffer
    glClearColor(0, 0, 0, 1); glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glm::ivec2 resolution = window->GetResolution();
    // sets the screen area where to draw
    glViewport(0, 0, resolution.x, resolution.y);
}

void AnimatieTransformari3D::Update(float deltaTimeSeconds)
{
    modelMatrix = glm::mat4(1);
    modelMatrix *= Transform3D::RotateOX(angularStepOX);
    modelMatrix *= Transform3D::RotateOY(angularStepOY);
    modelMatrix *= Transform3D::RotateOZ(angularStepOZ);
    modelMatrix *= Transform3D::Scale(scaleX, scaleY, scaleZ);
    modelMatrix *= Transform3D::Translate(translateX, translateY, translateZ);
    RenderMesh(meshes["box"], shaders["VertexNormal"], modelMatrix);
}

void AnimatieTransformari3D::FrameEnd()
{
    DrawCoordinatSystem();
}
```

Implementarea transformarilor 3D – exemplu (3)

```
void AnimatieTransformari3D::OnInputUpdate(float deltaTime, int mods)
{
    //apelata la fiecare deltaTime secunde
    if (window->KeyHold(GLFW_KEY_1)) angularStepOX += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_2)) angularStepOX -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_3)) angularStepOY += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_4)) angularStepOY -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_5)) angularStepOZ += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_6)) angularStepOZ -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_7)) scaleX += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_8)) scaleX -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_9)) scaleY += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_0)) scaleY -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_MINUS)) scaleZ -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_EQUAL)) scaleZ += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_UP)) translateY += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_DOWN)) translateY -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_LEFT)) translateX -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_RIGHT)) translateX += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_PAGE_UP)) translateZ += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_PAGE_DOWN)) translateZ -= step * deltaTime;
}
```

Implementarea transformărilor 3D – exemplu (4)

```
void AnimatieTransformari3D::OnKeyPress(int key, int mods)
{
    if (key == GLFW_KEY_SPACE)
    { //initializare
        angularStepOX = 0; angularStepOY = 0; angularStepOZ = 0;
        scaleX = 1; scaleY = 1; scaleZ = 1;
        translateX = 0; translateY = 0; translateZ = 0;
    }
}
```